

**AI-BASED TOMATO RIPENING STAGE
DETECTION AND SORTING SYSTEM WITH
SHELF-LIFE ESTIMATION FOR THE COMMONLY
CONSUMED PADMA VARIETY IN SRI LANKA
USING COMPUTER VISION AND CONVEYOR
AUTOMATION**

A dissertation submitted to the
Department of Computer Science and Informatics,
Faculty of Applied Sciences,
Uva Wellassa University
in partial fulfilment of the requirements for the award of the
Degree of Bachelor of Science (Honours) in Industrial Information Technology

by

**WITHARAMAGE KANUSHKA GAYAN
PALLAMULLE LEKAMLAGE HASINDU HIRUSHAN
KARNASOORIYA RAGALAGE DINUSHI HANSANI
GUNAWARDHANA**

Department of Computer Science and Informatics

Faculty of Applied Sciences

Uva Wellassa University, Sri Lanka

August 2026

DECLARATION

We do hereby declare that the work reported in this dissertation was exclusively carried out by us under the supervision of Ms. D.P. Jayathunga, Dr. U.G.A.T. Premathilake and Ms. M.P.A.M. Rathnakumara. It describes the results of our own independent research except where due reference has been made in the text. No part of this dissertation has been submitted earlier or concurrently for the same or any other degree.

Date

Dinush

K.R.D.H. Gunawardhana

(UWU/IIT/21/017, iit21017@uwu.ac.lk)

Hirushan

P.L.H. Hirushan

(UWU/IIT/21/019, iit21019@uwu.ac.lk)

Gayan

W.K. Gayan

(UWU/IIT/21/062, iit21062@uwu.ac.lk)

We endorse the declaration by the candidate(s).

Ms. D.P. Jayathunga

Dr. U.G.A.T. Premathilake

Date

Date

Ms. M.P.A.M. Rathnakumara

Date

Acknowledgement

We wish to express our sincere gratitude to our principal supervisor, Ms. D.P. Jayathunga, and to our co-supervisors, Dr. U.G.A.T. Premathilake and Ms. M.P.A.M. Rathnakumara, of the Department of Computer Science and Informatics, Uva Wellassa University, for their guidance, constructive criticism and continued encouragement throughout every stage of this research.

We thank the academic and technical staff of the Department of Computer Science and Informatics and of the Faculty of Applied Sciences for their instruction and for access to the facilities used in this work. We are grateful to the management and staff of the tomato collecting centre in Bandarawela, Badulla District, for permitting the collection and photography of Padma tomato samples over the course of the study.

Finally, we thank our families and colleagues for their patience and support during the completion of this dissertation.

Contents

Declaration	i
Acknowledgement	iii
Contents	iv
Abstract	viii
List of Figures	ix
List of Tables	xi
List of Abbreviations	xiii
List of Abbreviations	xiii
1 Introduction	1
1.1 Background of the Study	1
1.2 Problem Identification	2
1.2.1 Research Questions	3
1.2.2 Relevance and Significance of the Study	3
1.3 Research Objectives	4
1.3.1 Aim	4
1.3.2 Specific Objectives	4
1.4 Expected Outcomes	5

2	Literature Review	6
2.1	Theoretical Background	6
2.1.1	Ripening Physiology and Cultivar-Specific Grading	6
2.1.2	Real-Time Single-Stage Object Detection	7
2.1.3	Learned Imaging Conditions as a Misleading Cue	8
2.1.4	Conveyor Actuation and Control	9
2.1.5	Embedded Integration and Wireless Command Transfer	9
2.1.6	Shelf-Life Estimation and Operational Indicators	10
2.2	Gaps in Knowledge	10
3	Materials and Methods	12
3.1	Introduction	12
3.2	Overall Research Methodology	12
3.3	Research Materials	13
3.3.1	Tomato Samples	13
3.3.2	Hardware Components	14
3.3.3	Software and Technologies	15
3.4	Sampling Method and Sample Collection	15
3.5	Image Acquisition	15
3.6	Dataset Description	16
3.7	Ripening Stage Classification Criteria	18
3.8	Image Annotation	19

3.8.1	Annotation Quality Control and Correction	20
3.9	Image Preprocessing and Augmentation	20
3.10	Dataset Splitting	21
3.11	Model Development	22
3.12	Model Training	23
3.13	Validation and Evaluation Procedure	23
3.14	Real Time Detection Process	24
3.15	Hardware Prototype Development	25
3.16	Software System Development	27
3.17	Hardware and Software Integration	27
3.18	Shelf Life Observation Procedure	28
3.19	Complete End to End System Process	29
3.20	Experimental Procedure	30
3.21	System Testing Procedure	31
3.22	Ethical and Safety Considerations	32
4	Results and Discussion	33
4.1	Detection Performance on the Held-Out Test Split	33
4.2	Effect of the Dataset Corrections	35
4.3	Live Classification Validation	36
4.4	Conveyor Transport and End-to-End Sorting	39
4.5	Shelf-Life Observation	39

4.6	System Test Outcomes	40
4.7	Discussion	41
5	Conclusion	44
5.1	Summary of Findings	44
5.2	Achievement of Objectives	45
5.3	Limitations	45
5.4	Recommendations for Future Work	46
5.5	Concluding Remarks	46
	References	51
	Appendix	52

Abstract

Many tomatoes are lost after harvest in Sri Lanka because the fruit is graded by hand. Hand grading is slow, differs from one worker to another, and bruises the fruit. This research built and tested a real-time system that uses a camera and a moving belt to grade and sort tomatoes of the local Padma variety (*Solanum lycopersicum*) into four ripening stages and a defect group.

A dataset of 9,000+ images was collected at a commercial collecting centre in Bandarawela. Each fruit was labelled as one of five classes: green, breaker, turning, red or defect. The images were split 70:20:10 into training, validation and held-out test sets. A YOLOv8-medium detector was trained on this dataset and tested once on the held-out set. It reached a precision of 0.906, a recall of 0.922 and a mAP@50 of 0.936, and it processed each frame in 6.7 ms. To give one class per tomato, the system tracks each fruit across frames and takes a confidence-weighted vote.

A conveyor prototype was built with a stepper-driven belt, an overhead camera, four infrared sensors and four servo gates, all controlled by an ESP32 board. Each gate opens only when the fruit reaches that gate's own sensor. In live trials all five classes were classified correctly, the commands reached the hardware, and the gates sorted the four ripening classes while defect fruit passed to a reject stream.

A 31-day study of 50 green *Padma* tomatoes gave a remaining shelf life for each class: green 31, breaker 29, turning 24, red 12 and defect 0 days. These values agreed closely with an earlier published study. Every result is saved to a database and shown on a live dashboard with throughput, class share, defect ratio and mean shelf life. The work shows that low-cost hardware and a single camera can grade this local variety and give a packhouse both a physical sort and batch-level information.

Keywords: *Padma tomato; YOLOv8; post-harvest sorting; conveyor automation; shelf-life estimation*

List of Figures

1.1	Tomato cultivation areas in the Badulla District and major local cultivars.	2
3.1	Overall research methodology	13
3.2	Image acquisition setup for dataset capture and the conveyor mounted camera arrangement	16
3.3	Representative sample images for each of the five target classes	17
3.4	Example of bounding box annotation for a tomato image	19
3.5	Real time tomato detection and classification process	25
3.6	Computer aided design model and assembled hardware prototype	26
3.7	Software architecture and operator dashboard of the detection and sorting application	27
3.8	Complete end to end system workflow	29
4.1	Training and validation curves for the deployed model	34
4.2	Detection metric curves as a function of confidence threshold: precision (top left), recall (top right), F1 (bottom left) and precision–recall (bottom right)	34
4.3	Real-time dashboard detections for each of the five classes during the August 2026 system trials: green, breaker, turning, red and defect (left to right), each shown with its bounding box, predicted class and confidence while the tracker is in the TRACKING state	37
4.4	Console log output of the real-time sorting pipeline, showing the classification of a green tomato, speed-scaled gate actuation scheduling, wireless transmission of the class1 command, and subsequent queue-state updates acknowledged (ACK) by the ESP32 microcontroller	38

5.1	Schematic wiring diagram of the ESP32 control system, illustrating GPIO pin configurations, sensor input interfaces, servo/stepper actuator connections, and isolated power distribution domains.	60
-----	---	----

List of Tables

2.1	Post-harvest stage durations reported for the Padma variety under simulated market conditions	7
3.1	Materials and samples used in the research	14
3.2	Principal hardware components of the conveyor sorting prototype . . .	14
3.3	Software, frameworks and libraries used in the research	15
3.4	Composition of the dataset used for model development	17
3.5	Per class annotated object instance counts by dataset split	17
3.6	Correspondence between the six standard maturity stages and the four operational classes used in this research	18
3.7	Ripening stage and quality classes used in this research	19
3.8	Dataset split used for model development	22
3.9	Training configuration used for model development	23
3.10	System test cases and expected results	31
4.1	Held out test set performance of the deployed YOLOv8m model	33
4.2	Defect class and overall detection performance across successive dataset corrections	35
4.3	Live dashboard class distribution before and after the imaging-conditions correction	36
4.4	Real-time pipeline output for one representative fruit of each class in the August 2026 five-class trial, from the dashboard and controller console logs	38
4.5	Recorded stage durations and derived remaining shelf life by operational class	40

4.6	System test outcomes	41
5.1	Final per-class annotated instance counts by dataset split	52
5.2	Full training configuration for the deployed model	54
5.3	Best and last logged training epochs, validation split (deployed model) .	55
5.4	ESP32 pin assignment, currently deployed firmware	57
5.5	Command protocol of the currently deployed firmware	61

List of Abbreviations

AI	Artificial Intelligence
ANN	Artificial Neural Network
CAD	Computer Aided Design
CNN	Convolutional Neural Network
COCO	Common Objects in Context
CUDA	Compute Unified Device Architecture
DFL	Distribution Focal Loss
FIFO	First-In, First-Out
FN	False Negative
FP	False Positive
FPS	Frames Per Second
GPU	Graphics Processing Unit
HSV	Hue, Saturation, Value
IoU	Intersection over Union
IR	Infrared
KPI	Key Performance Indicator
LED	Light Emitting Diode
mAP	Mean Average Precision
MiB	Mebibyte
NIR	Near Infrared
R-CNN	Region-based Convolutional Neural Network
RGB	Red, Green, Blue
RPN	Region Proposal Network

RT-DETR	Real-Time Detection Transformer
SPP	Serial Port Profile
SVM	Support Vector Machine
TP	True Positive
USB	Universal Serial Bus
USDA	United States Department of Agriculture
VRAM	Video Random Access Memory
YOLO	You Only Look Once

Introduction

1.1 Background of the Study

Post-harvest loss is a major source of inefficiency in the supply of perishable fresh produce, and it falls heaviest on growers in regions where cold-chain infrastructure is limited and grading is performed by hand. The tomato (*Solanum lycopersicum*) is particularly exposed. It is a climacteric fruit, so respiration and ethylene production continue after harvest and drive a rapid sequence of chlorophyll loss, carotenoid accumulation and cell-wall softening; once softening is advanced the fruit bruises easily and becomes vulnerable to fungal decay (Abekoon et al. 2024).

In Sri Lanka the locally recommended Padma variety is an important cash crop for smallholders in the Badulla, Nuwara Eliya, Kandy and Matale districts, valued for its resistance to bacterial wilt and its keeping quality (Abekoon et al. 2024). Fruit reaching collecting centres in these districts is still graded visually by hand. Operators sort bulk batches by the proportion of red surface colour into categories such as green, breaker, turning and red. Manual inspection of this kind is subjective, varies between operators and across a working shift, and requires repeated handling of every fruit. Two consequences follow. Misgraded overripe fruit entering a packed batch accelerates the ripening of its neighbours, and the repeated handling itself causes microscopic skin damage that provides an entry route for decay organisms (Abekoon et al. 2024).

Automated grading offers a route around both problems. Convolutional neural networks, and in particular single-stage detectors of the You Only Look Once (YOLO) family, now classify produce accurately and quickly enough to run on a moving conveyor (Moya et al. 2025, Terven & Cordova-Esparza 2023), and when coupled to electronic actuators they can grade and divert fruit without a human touching it (Geetha et al. 2025). Applying such a system to a specific local cultivar is not, however, a matter of deploying an existing model. It requires a dataset built for that cultivar, a decision

policy suited to the way the fruit actually presents itself on a belt, and mechanical and control design that diverts fruit reliably without bruising it.

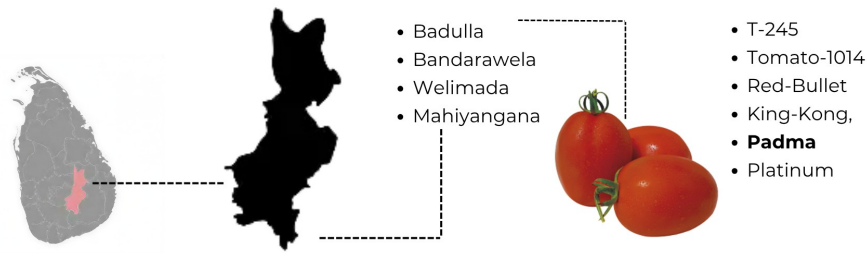


Figure 1.1: Tomato cultivation areas in the Badulla District and major local cultivars.

1.2 Problem Identification

The post-harvest handling of Padma fruit presents three linked problems.

At the biological level, the fruit softens quickly once ripening begins and is easily bruised by manual handling, so every additional handling step shortens the period during which the fruit remains saleable (Abekoon et al. 2024).

At the computational level, published ripeness models are trained predominantly on other cultivars, on other datasets and under other imaging conditions. Models built for one cultivar transfer poorly to another whose size, surface texture and colour progression differ; Nalupano et al. (2024) showed this directly when a general architecture had to be retrained specifically for the Philippine Kinalabasa variety. A further and less frequently reported difficulty is that a detector may learn the imaging conditions of a class rather than the class itself, so that a model performing well on an offline test split fails on live camera input. No public dataset exists for Padma fruit annotated for object detection across four ripening stages together with a defect class.

At the system level, most work reports a model evaluated offline on static images and stops there. Where a physical sorting rig is built, gate actuation is commonly timed from an assumed constant belt speed, which accumulates error as belt speed, load and drive slip vary. Furthermore, a grading decision by itself tells a packhouse manager only what a fruit is now, not how long it will remain saleable, and gives no batch-level view of the line.

1.2.1 Research Questions

1. How accurately can a single-stage object detector localise and classify Padma fruit into four ripening stages and a defect class from a purpose-built, cultivar-specific dataset, and which architecture within that family performs best on this task?
2. What decision policy converts a stream of per-frame detections into one reliable classification per fruit as it travels beneath a camera, given that the ripening classes lie on a continuous colour gradient?
3. What control architecture synchronises classification with physical gate actuation on a conveyor without relying on an assumed constant belt speed?
4. How can measured post-harvest stage durations for Padma fruit be converted into a remaining shelf-life value attached to each graded fruit, and reported alongside line-level operating indicators on a live dashboard?

1.2.2 Relevance and Significance of the Study

There is a clear economic case for this work. Manual grading requires repeated human inspection, which is slow, costly, and inconsistent different inspectors judge ripeness differently, and this inconsistency lets overripe fruit slip into packed batches (Abekoon et al. 2024). Replacing repeated manual checks with a single automated pass reduces both handling damage and this inspector-to-inspector inconsistency.

This study makes three scientific contributions.

First, it produces a new dataset: annotated images of the Padma tomato cultivar, labelled across four ripening stages plus a defect class. No such dataset currently exists.

Second, it does not treat detection accuracy as just an offline number. Instead, it builds and tests a real, working sorting machine that uses the detector’s output to physically sort fruit, triggered by sensors. Reporting this failure, and how it was corrected, is itself a useful contribution, since this type of mistake is invisible in ordinary offline test scores and only shows up when the system is tested in real, live conditions.

Third, the study links each grading decision to a remaining shelf-life estimate for this specific cultivar, and shows both the grade and the shelf-life estimate together on an operator dashboard. This means the system's output is not just a physical sort into bins it also gives the operator useful logistical information, namely how long each batch can still be stored or sold.

1.3 Research Objectives

1.3.1 Aim

To design, develop and evaluate an integrated, real-time detection and mechatronic sorting system for the Sri Lankan Padma tomato variety, which grades fruit into four ripening stages and a defect class, diverts it physically, associates each graded fruit with a measured remaining shelf life, and reports line-level operating indicators on a live dashboard.

1.3.2 Specific Objectives

1. To compile, annotate and preprocess a cultivar-specific image dataset representing four ripening stages (green, breaker, turning, red) and a defect class for the Padma variety.
2. To train and evaluate a YOLOv8 object detector on this dataset against a held-out test split, and to compare it under identical data conditions with a YOLO26-medium model in order to justify the deployed architecture on evidence rather than assertion.
3. To measure the duration of each ripening stage for Padma fruit under room-temperature storage, and to derive from those durations a per-class remaining shelf-life value applied at the moment of classification.
4. To design and construct a conveyor prototype controlled by an ESP32 microcontroller, in which gate actuation is released by an Infrared (IR) presence sensor at each gate rather than by an elapsed time computed from belt speed.

5. To develop a live dashboard that reports throughput, class distribution, defect ratio and mean remaining shelf life from the logged detection record.

1.4 Expected Outcomes

1. An annotated Padma-specific detection dataset covering four ripening stages and a defect class, available to support further localised research.
2. A trained model for Padma ripening stage and defect condition, with performance reported on a held-out test split and against a comparison architecture.
3. A working conveyor sorting prototype using non-contact infrared triggering and servo-actuated diverter gates.
4. A per-class remaining shelf-life mapping derived from measured stage durations for this cultivar.
5. A live dashboard reporting throughput, class distribution, defect ratio and mean remaining shelf life over the logged detection record.

Literature Review

2.1 Theoretical Background

An automated grading and sorting machine draws on three areas: how a tomato ripens, which decides what is being graded; the detection method that does the grading; and the control design that turns a grading decision into a physical action. Each is reviewed below.

2.1.1 Ripening Physiology and Cultivar-Specific Grading

A tomato keeps ripening after it is picked. The plant hormone ethylene makes the skin lose its green colour while red pigments, mainly lycopene, build up, so the surface changes from all green to deep red. At the same time the cell walls break down and the fruit softens (Abekoon et al. 2024). Once the fruit is soft it bruises easily, and bruised or broken skin lets decay organisms in, so soft fruit is fragile for selling.

Because the surface colour follows this internal change, colour is what visual grading is based on. The United States Department of Agriculture (USDA) standard sets six stages by the share of red skin: green; breaker (up to 10 %); turning (10–30 %); pink (30–60 %); light red (60–90 %); and red (over 90 %) (United States Department of Agriculture 1991). These stages were made for grading and reporting by eye, not for a camera. The two smallest bands fall exactly where a camera is weakest: with pack-house lighting, a moving belt and shine off the glossy skin, telling 8 % red from 12 % red is not a reliable measurement.

General standards also do not carry well from one variety to another. Nalupano et al. (2024) had to retrain a MobileNetV2 network just for the Philippine Kinalabasa variety; it reached 94 % accuracy but only 64 % specificity, which shows both that variety-specific training is needed and that a high headline accuracy can hide poor separation

between classes. For Sri Lanka, Abekoon et al. (2024) studied the Padma variety directly. They kept 100 greenhouse-grown fruits, 25 from each of four districts, in a room set to the average conditions of sixty Sri Lankan markets (29 °C, 80 % relative humidity, 810 lux), and photographed them every day until they spoiled, giving 12,342 images. Their stage lengths are shown in Table 2.1. That study gives both a colour-to-time relationship for this variety and the six-stage frame that the present study builds its own classes on.

Table 2.1: Post-harvest stage durations reported for the Padma variety under simulated market conditions

Maturity stage	Days in stage	Post-harvest day range
Green	2	0–2
Breakers	2	3–4
Turning	3	5–7
Pink	4	8–11
Light red	8	12–19
Red	12	20–31
Total	31	

Source: Abekoon et al. (2024), Table 4. Conditions: 29 °C, 80 % relative humidity, 810 lux; 100 fruits.

2.1.2 *Real-Time Single-Stage Object Detection*

Early farm vision systems used fixed colour and texture rules with simple classifiers. Santoso et al. (2024) compared RGB, HSV and CMYK colour spaces with a correlation-matching method and found CMYK separated ripeness best, with a correlation of 0.87. Liu et al. (2019) combined shape-gradient features with a Support Vector Machine (SVM) and a colour rule to remove false detections. Garcia et al. (2019) judged ripeness from simple pixel-colour statistics. These methods are light to run, but they rely on features chosen by hand and get worse when the light, shadows or background differ from the setup they were tuned on (Ifmalinda et al. 2023).

Convolutional neural networks removed the need to choose features by hand, learning them from the data instead. Allo et al. (2025) trained such a network on 3,600 public images across three ripeness classes, and Centino et al. (2025) reached 95.52 % on a ripe-or-unripe task after first removing the background and changing the colour space.

Waseem et al. (2025) did the same task on limited hardware, using a ResNet-18 network with pruning and quantisation to keep it fast. All three, though, label a whole image at once. A sorting line also needs to know *where* each fruit is, because one frame can hold several fruits and each needs its own decision.

Object detectors do this. Two-stage detectors such as Faster R-CNN first suggest regions with a Region Proposal Network (RPN) and then classify them; this is accurate but too slow for a moving belt. Single-stage detectors in the YOLO family instead predict the boxes and the class scores in one pass over the image (Redmon et al. 2016). YOLOv8 is anchor-free and combines image features at several scales efficiently, which gives the speed needed for real-time work (Terven & Cordova-Esparza 2023).

Most tomato-specific work adapts this family to hard viewing conditions. Yang et al. (2025) and Fan & Chai (2026) changed YOLOv8 to cope with hidden fruit and changing light in greenhouses and orchards; Huang et al. (2025) added a small-object detection head and multi-scale fusion to YOLOv10; Wang et al. (2026) and Ding et al. (2026) made lighter versions for small devices. Safira et al. (2026) compared YOLOv8 directly with a real-time detection transformer for tomato ripeness, and is the clearest published reason to prefer a YOLO detector for this task. Almost all of this work is for harvesting robots in the field or greenhouse, where the main problems are hidden fruit and busy backgrounds. A packhouse belt is the opposite case: a plain background and usually one fruit in view, but the decision must be made in the few seconds the fruit is visible and then carried out by a machine.

2.1.3 Learned Imaging Conditions as a Misleading Cue

A detector learns whatever tells its classes apart in the training images, even things that have nothing to do with the fruit. If one class was photographed under different light or against a different background from the rest, the network can learn that difference instead of the fruit feature it was meant to learn. It then looks accurate on a test set drawn from the same source, but fails on live input. The danger is greatest when a class is rare and is filled out with images from an outside source, which is exactly the case for a defect class. This is a known risk. The usual fixes are to keep the capture

conditions the same for every class, and to grow a small class with augmentation rather than with outside images (Shorten & Khoshgoftaar 2019, Buslaev et al. 2020). It is still rarely reported in applied farm-vision papers, which usually give only final scores and not the debugging history behind them. Section 3.8.1 describes a case of this found in the present study.

2.1.4 Conveyor Actuation and Control

Once a fruit is classified, its gate must open at the moment the fruit reaches it. The simple way is to work out a waiting time from the belt speed and the distance from the camera to the gate: the further the gate, the longer the wait. The weakness of this method is that it assumes the belt speed is constant and known. On a real rig the belt speed changes with tension, load and drive slip, and because the distance used is the whole camera-to-gate length, the timing error builds up along the line and is worst at the last gate.

The better way is to detect the fruit arriving instead of predicting it. A presence sensor is placed at each gate, and each classification waits in a queue until that gate's sensor is triggered. Existing rigs already use belt sensors, but mostly for another job: in Moya et al. (2025), sensors along the belt detect fruit and start the camera, while the sorting servos act on the classification result. Geetha et al. (2025) likewise link detection to a servo-driven sorting prototype. Releasing each gate from its own sensor, so that no timing calculation is left in the actuation path, is the design used here.

2.1.5 Embedded Integration and Wireless Command Transfer

Farm vision systems are usually split between a host computer that runs the model and a microcontroller that drives the actuators. Patria et al. (2025) used an ESP32-CAM with a small neural network on a mobile field robot, and Saxena et al. (2025) paired ESP32 sensor nodes with a Raspberry Pi running YOLOv8, using pruning and quantisation for a 35 % speed-up. Both show this split of work, though in both the microcontroller only senses and does not drive a sorting mechanism.

When the link between the host and the controller is wireless, a command may not arrive. A serial-over-Bluetooth link on a running rig can drop packets, and in sorting a lost command is not just a resend: it takes a fruit out of the queue and throws off every gate assignment after it. So the command protocol needs a sequence number on each command, a clear acknowledgement, and duplicate rejection, so that a resent command is acknowledged again but not acted on twice.

2.1.6 Shelf-Life Estimation and Operational Indicators

Shelf life is how long produce stays safe, firm and good to eat under set storage conditions (Tarlak 2023). The literature estimates it in two ways. The first measures it directly, by keeping a batch of fruit under set conditions and recording how long each stage lasts, as Abekoon et al. (2024) did for the Padma variety. The second learns it, as Geetha et al. (2025) did with deep models linked to a sorting prototype. There are also non-destructive instruments: Borba et al. (2021) used a portable Near Infrared (NIR) spectrometer to check tomato quality in the field, and Liu et al. (2024) combined several sensors for maturity. These read internal quality that a colour camera cannot, but need instruments a low-cost packhouse rig cannot carry.

Either way, the estimate is only useful when it is added up. A packhouse manager needs throughput, class share, defect ratio and mean remaining shelf life for a whole batch, not a figure for one fruit (Geetha et al. 2025).

2.2 Gaps in Knowledge

No conveyor-integrated detection system exists for the Padma variety. Abekoon et al. (2024) characterised Padma ripening and post-harvest duration thoroughly, but the work is offline classification of static images of single fruits, with no detector, no localisation and no physical sorting. Conversely, conveyor-integrated systems such as Moya et al. (2025) are calibrated for other cultivars in other production contexts. No published work reports a detection model trained on Padma fruit and evaluated on an operating sorting line, and no public Padma detection dataset spanning four ripening stages and a defect class exists to build one from.

Gate release is generally predicted rather than detected. Belt sensors are already used in sorting rigs, but predominantly to trigger image capture (Moya et al. 2025). Actuation itself is usually timed from the belt speed and the camera-to-gate distance, which makes sorting accuracy depend on the belt speed staying constant and lets the error grow along the line. A control architecture in which each gate is released by its own presence sensor, with classifications held in a queue until arrival, removes that dependency but is not established in the tomato sorting literature.

Grade-to-shelf-life mapping and line-level reporting are rarely combined with an operating rig. This gap is the narrowest of the three, and must be stated carefully. Geetha et al. (2025) do link deep-learning grading to shelf-life prediction and a servo-driven sorting prototype, so it cannot be claimed that no such system exists. What remains absent is a system in which the shelf-life values are measured for the specific cultivar being graded, rather than learned from generic data, and in which each classification and its associated remaining shelf life are logged to a persistent record that drives live batch-level operating indicators. It is that combination, for this cultivar, that the present study addresses.

Materials and Methods

3.1 Introduction

This chapter describes the development of the tomato ripening detection and sorting system. The research consists of two main parts: a computer vision component (image collection, model training, and testing) and an engineering component (fabrication of the conveyor belt, electronics, and software). Together, they form a conveyor prototype that detects a tomato on a moving belt, classifies its ripening stage or defect condition, estimates its remaining shelf life, and diverts it into the appropriate output bin.

This chapter is organized chronologically based on the research phases: materials, dataset preparation, model development, real-time detection, hardware and software development, integration, shelf-life observation, and system evaluation. Results are reported in Chapter 4; raw data and full component specifications are provided in the appendices.

3.2 Overall Research Methodology

This allowed for revisions to the dataset, annotation rules, and AI model whenever evaluations identified issues. The sequential stages were: problem definition; sample collection and image acquisition; dataset preparation, annotation, and splitting; preprocessing and augmentation; model training and testing; dataset correction and retraining; hardware design and fabrication; software development; hardware and software integration; real-time detection and sorting; and system testing. This workflow, including the feedback correction loop, is illustrated in Figure 3.1.

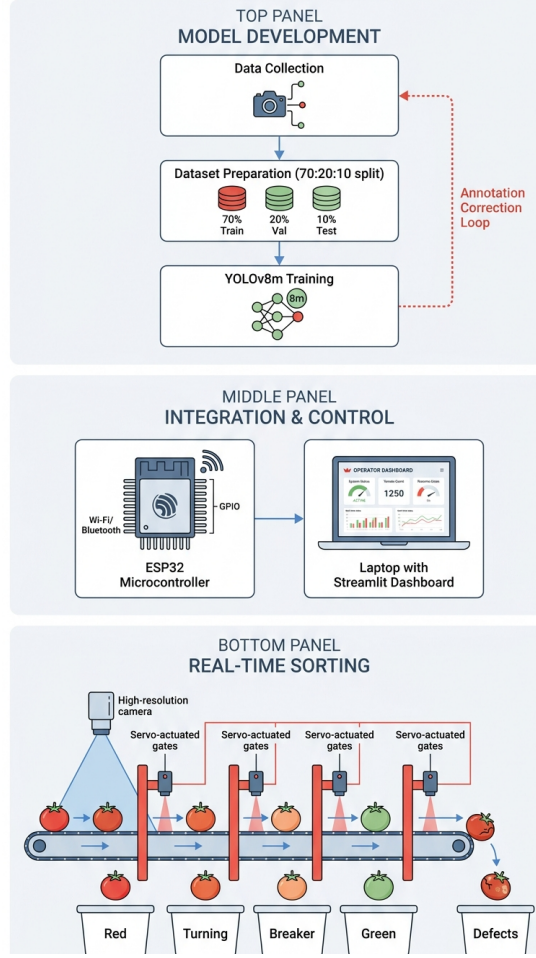


Figure 3.1: Overall research methodology

3.3 Research Materials

3.3.1 Tomato Samples

Padma variety tomatoes were acquired from a commercial collecting center in Bandarawela, Badulla District, which receives fruit from multiple regional farms. Sourcing from a multi-farm center rather than a single farm ensures natural variation in size, texture, maturity, and handling damage. Fruits were visually sorted during capture into the five categories defined in Section 3.7. The materials used are listed in Table 3.1.

Table 3.1: Materials and samples used in the research

Material	Description	Purpose
Padma tomato fruit	Fresh fruit at four ripening stages, and fruit with visible cracking, bruising or rot, from a commercial collecting centre, Bandarawela	Image dataset, shelf-life observation and conveyor testing
Publicly sourced defect images	Third-party defect images used in an earlier dataset version, captured under uncontrolled backgrounds and lighting	Removed during quality control (Section 3.8.1)

3.3.2 Hardware Components

The prototype consists of a conveyor belt equipped with a fixed overhead camera and four servo-actuated sorting gates. Its primary components are detailed in Table 3.2; the complete pin assignment and wiring diagrams are provided in Appendix C.

Table 3.2: Principal hardware components of the conveyor sorting prototype

Component	Model / specification	Qty
Microcontroller	ESP32 development board (DOIT DEVKIT V1); runs the sorting firmware	1
Drive motor	NEMA 23 stepper motor	1
Motor driver	DM542 step-and-direction driver	1
Power supplies	24 V 5 A driver rail; 12 V 2 A logic rail	1 each
Gate actuators	MG995 servo with printed diverter blade, one per ripening class	4
Presence sensors	Infrared reflective sensor, one at each gate	4
Power transmission	Motor pulley, roller pulley and timing belt (3:1 reduction)	1 set
Frame and mounts	25 mm box-section frame, bearing assemblies, printed brackets, adjustable camera mount	1 set
Speed control	Potentiometer read by the microcontroller	1
Camera	USB webcam, 960×720 pixels, 30 FPS	1
Host computer	Workstation with NVIDIA GeForce RTX 3070 Ti, 8,192 MiB VRAM	1

3.3.3 Software and Technologies

The software environment is listed in Table 3.3.

Table 3.3: Software, frameworks and libraries used in the research

Software	Purpose
Python, PyTorch/torchvision, Ultralytics (YOLOv8)	Model training, validation and inference
OpenCV	Camera capture, frame processing, box rendering
Albumentations	Offline bounding-box-aware augmentation
Streamlit and Plotly	Operator dashboard and charts
SQLite	Local detection and indicator database
Roboflow	Annotation and versioned dataset export
PlatformIO, Arduino framework, ESP32Servo	Firmware build, upload and servo control
SolidWorks and Onshape	Three-dimensional design of the prototype

3.4 Sampling Method and Sample Collection

Images were collected using purposive sampling rather than random sampling: fruits were deliberately photographed across all five target categories to ensure that the less common defect condition was adequately represented (Etikan et al. 2016).

Data collection commenced in January 2026. All dataset images were captured with the fruit placed against a white background inside a lightbox under LED illumination. This maintained consistent background and lighting conditions across all sessions and classes. This consistency was crucial: as explained in Section 3.8.1, images captured under variable lighting conditions caused the model to learn the background lighting instead of the fruit characteristics.

3.5 Image Acquisition

The real-time deployment setup utilizes a different camera configuration than the dataset capture process. For real-time inference, the system uses a USB webcam mounted

above the conveyor belt, operating at 960×720 pixels and 30 FPS. Upon initialization, the camera’s automatic exposure and white balance are allowed approximately 1.5 seconds to stabilize before any frame is sent to the model. Frames captured during this initial period can exhibit severe color distortions that may alter the predicted class, and are therefore discarded.

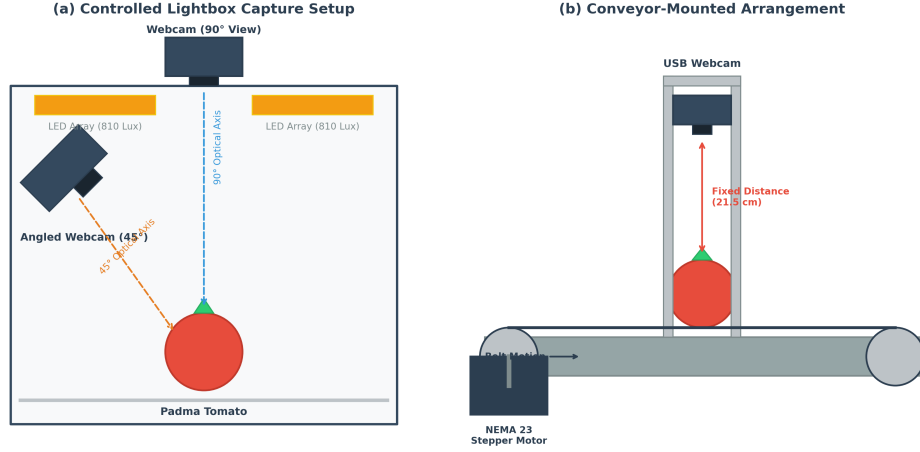


Figure 3.2: Image acquisition setup for dataset capture and the conveyor mounted camera arrangement

3.6 Dataset Description

The dataset was compiled and versioned using Roboflow (Dwyer et al. 2024) and exported in the YOLOv8 label format. Its composition is detailed in Table 3.4, and the per-class instance counts are provided in Table 3.5. Instance counts exceed image counts because a single image may contain multiple fruits. The images added to augment the defect class (Section 3.9) are listed separately to prevent the mixing of synthetic and captured data in the metrics.

Table 3.4: Composition of the dataset used for model development

Property	Value
Number of classes	5 (breaker, defect, green, red, turning)
Training images	5,498 captured, plus 668 offline-augmented defect images
Validation images	1,574
Test images (held out)	777
Total distinct captured images	7,849
Annotation format	YOLO normalised bounding box: class index, x centre, y centre, width, height
Model input resolution	640×640 pixels after resizing

Table 3.5: Per class annotated object instance counts by dataset split

Class	Train (captured)	Train (augmented)	Validation	Test	Total
breaker	1,137	0	326	191	1,654
defect	448	896	137	66	1,547
green	1,380	0	429	191	2,000
red	1,434	0	395	190	2,019
turning	1,138	0	309	149	1,596
Total	5,537	896	1,596	787	8,816

The defect class contains only 66 test instances, compared to 149–191 for each ripening class. Consequently, it is treated with increased statistical caution in the evaluation approach detailed in Section 3.13.

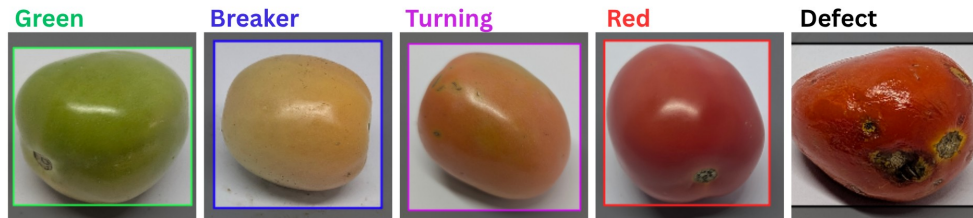


Figure 3.3: Representative sample images for each of the five target classes

3.7 Ripening Stage Classification Criteria

Five classes were established: four ripening stages and one defect class. Defect is treated as a separate class rather than an attribute of a ripening stage, because damaged fruit is handled similarly regardless of its color.

The four ripening stages were defined by merging adjacent pairs of the six standard USDA maturity stages (United States Department of Agriculture 1991), aligning with the methodology used by Abekoon et al. (2024) for this tomato variety. The four classes correspond directly to the four sorting gates. Furthermore, the two narrowest standard bands occur where a camera has the most difficulty estimating the red surface percentage. Merging these bands creates broader boundaries that the model can reliably distinguish on a moving conveyor belt. This correspondence is outlined in Table 3.6, and the resulting criteria are shown in Table 3.7.

Table 3.6: Correspondence between the six standard maturity stages and the four operational classes used in this research

Standard stage	Standard red surface	Operational class	Operational red surface
Green	0 %	Green	0 %
Breaker Turning	≤ 10 % 10–30 %	Breaker	< 30 %
Pink Light red	30–60 % 60–90 %	Turning	30–80 %
Red	> 90 %	Red	> 80 %

Standard stages and boundaries from United States Department of Agriculture (1991). The operational upper boundary of the turning class is set at 80 % rather than 90 % so that fully coloured fruit falls into the red class under belt lighting.

Table 3.7: Ripening stage and quality classes used in this research

Class	Stage	Identification characteristics
Green	Unripe (Stage 1)	Entirely green skin, no red or orange colouration
Breaker	Onset of ripening (Stage 2)	Green-to-yellow transition, less than 30 % red or pink
Turning	Mid ripening (Stage 3)	30–80 % of the surface red, mixed green/orange/red patches
Red	Fully ripe (Stage 4)	More than 80 % of the surface red, uniform ripe colouration
Defect	Rejected quality	Visible cracking, bruising or rot; can occur at any stage, receives no ripening label

3.8 Image Annotation

All images were annotated using Roboflow (Dwyer et al. 2024) and exported in the YOLOv8 format. In this format, each image includes a corresponding label file containing one line per annotated fruit: a class index followed by the bounding box’s normalized position and dimensions.

Every fruit visible in an image was assigned an individual bounding box and a single class label. For the defect class, the bounding box encompasses the entire fruit, rather than just the blemished area, and is labeled as defective regardless of the fruit’s underlying ripeness color. Maintaining a consistent bounding box style across all classes is essential, as the model learns the typical spatial dimensions for each class in addition to its visual features.

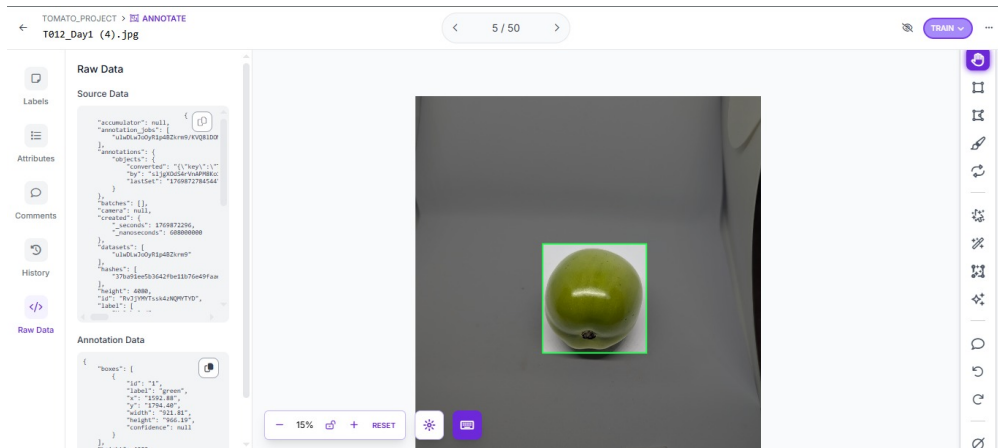


Figure 3.4: Example of bounding box annotation for a tomato image

3.8.1 Annotation Quality Control and Correction

The annotation process included a review and correction cycle. Two major corrections influenced the final dataset:

- **Inconsistent box style.** Defect images sourced from a public dataset were originally annotated with bounding boxes around only the blemished areas, whereas the newly captured project images annotated the entire fruit. Every such image was re-annotated using the whole-fruit style to ensure geometric consistency across all five classes.
- **Lighting acting as a false class cue.** Even after standardizing the bounding box style, live camera tests revealed that healthy breaker, turning, and red fruits were incorrectly classified as defective. This occurred because most defect images originated from the public dataset, which featured different backgrounds and lighting compared to the other four classes captured inside the lightbox. Consequently, the model incorrectly learned the lighting conditions, rather than the physical damage, as the indicator of a defect. The solution was to remove the mismatched public images, capture and label new defect images under the identical lightbox conditions, and apply the offline augmentation described in Section 3.9 to compensate for the reduced number of authentic defect photos. The dataset presented in Section 3.6 represents this corrected version.

The detailed annotation procedure is provided in Appendix A.

3.9 Image Preprocessing and Augmentation

All images were resized to 640×640 pixels for both training and inference, which is the standard input resolution for this architecture. During training, images undergo real-time augmentation. Vertical flipping was excluded, as fruit on a conveyor belt is never oriented upside down. This augmentation is standard practice for improving model generalization from limited data (Shorten & Khoshgoftaar 2019), and was utilized here

because real-world conveyor lighting differs from the lightbox conditions used during dataset capture.

Following the corrections outlined in Section 3.8.1, the defect class contained significantly fewer real training images than the ripening classes. Therefore, its training images were supplemented offline using the Albumentations library (Buslaev et al. 2020). Transformations including flip, rotation, scale, brightness, and hue changes were applied to generate two augmented copies of each defect training image. These additional images were utilized exclusively for training; the validation and test sets consisted solely of originally captured images.

An automated validation script executes before every training session and halts the process if structural errors are detected. It verifies the class list, confirms the existence of a test split, counts images and instances per class, flags missing or empty label files as well as duplicate images, and identifies any class with unusually small bounding boxes compared to the others. This final check was implemented following the box-style issue discussed above, ensuring that inconsistent box conventions do not enter the training phase unnoticed.

3.10 Dataset Splitting

The dataset was divided into training, validation, and test sets at the time of export, using the proportions detailed in Table 3.8 (a standard 70:20:10 split). The training set is used to fit the model parameters. The validation set is evaluated after every epoch to select the optimal model checkpoint. Finally, the test set is used only once, at the conclusion of the study, to measure the final unbiased performance.

Table 3.8: Dataset split used for model development

Subset	Images	Percentage	Role
Training	5,498	70.0 %	Fitting model parameters; supplemented with 668 augmented defect images
Validation	1,574	20.1 %	Monitoring generalisation, early stopping, checkpoint selection
Test	777	9.9 %	Held out from training; used once for final evaluation
Total	7,849	100 %	

3.11 Model Development

The system employs YOLOv8, an object detection architecture that localizes and classifies objects in a single pass over the image, eliminating the need for a separate region proposal step (Redmon et al. 2016, Terven & Cordova-Esparza 2023). The medium-sized YOLOv8m variant was utilized. The model was initialized with publicly released pre-trained weights and subsequently fine-tuned on the project’s custom dataset.

This architecture is well-suited for the task: a single camera frame above the conveyor may contain multiple fruits at different ripening stages. The sorting logic requires a bounding box, a class label, and a confidence score for each fruit, and it must process this information fast enough to match the conveyor belt speed. Published literature supports this choice for tomato detection (Safira et al. 2026). Furthermore, the medium variant optimally balances accuracy with the memory constraints of the host GPU. No separate classification step is required post-detection; the sorting decision relies directly on the detector’s prediction, which is aggregated across frames as detailed in Section 3.14.

A second detector (YOLO26m) was also trained using the same dataset. This allowed the chosen architecture to be validated using experimental evidence from this specific project, rather than relying solely on published literature. This comparison is detailed as Experiment 2 in Section 3.20.

3.12 Model Training

Training was conducted locally on a workstation equipped with an NVIDIA GeForce RTX 3070 Ti GPU (8,192 MiB VRAM). Due to the memory limitations of this GPU, the training script automatically selects an appropriate batch size and verifies available memory before initiating each run. The configuration used is summarized in Table 3.9; the complete configuration file is provided in Appendix B.

Table 3.9: Training configuration used for model development

Parameter	Value
Base architecture	YOLOv8m, initialised from pre-trained weights
Training data	5,498 captured images and 668 offline-augmented defect images
Maximum epochs and early stopping	150 epochs
Batch size and input size	Automatic selection; 640×640 pixels
Initial learning rate	0.01
Augmentation	Colour jitter; rotation, translation, scale; horizontal flip only; mosaic, mixup and random erasing

The training process consists of five steps: verifying that the framework, hardware device, and memory are prepared; executing the dataset audit described in Section 3.9, which aborts the run if anomalies are detected; training the model using the aforementioned settings while monitoring validation performance after each epoch to save the best checkpoint; applying early stopping when performance ceases to improve, retaining the best checkpoint as the final model; and finally, performing a single evaluation pass of that checkpoint on the held-out test set.

3.13 Validation and Evaluation Procedure

The validation set was evaluated after every epoch during training to prevent overfitting early, utilizing a fixed stopping rule rather than manual observation. The test set was evaluated only once. Therefore, the results reported in Chapter 4 represent an unbiased estimate rather than values the model was optimized to achieve.

Performance is measured using standard object detection metrics (Padilla et al. 2020): precision (the proportion of correct positive predictions), recall (the proportion of actual target instances successfully identified), an F_1 score that balances precision and recall (used to select the deployment confidence threshold), and mean average precision evaluated at a standard overlap threshold (mAP@50) and a stricter threshold that accounts for bounding box accuracy (mAP@50–95). A confusion matrix is also generated to illustrate misclassification trends between classes. As noted in Section 3.8.1, this diagnostic tool identified issues that summary statistics alone did not reveal.

Because the test set is limited in size particularly the defect class, which contains only 66 instances confidence interval is calculated for each reported metric using statistical resampling.

System-level metrics such as total fruits processed, class distribution, defect ratio, throughput, and average shelf life are aggregated directly from the detection logs.

3.14 Real Time Detection Process

During real-time operation, the system executes the following steps for every frame:

1. **Frame capture.** A live video frame is captured from the webcam at 960×720 pixels and 30 FPS.
2. **Preprocessing.** The frame is resized to the 640×640 pixel model input dimension.
3. **Inference.** The frame is processed through the trained model at a confidence threshold of 0.45, which represents the optimal balance between precision and recall.
4. **Tracking and vote accumulation.** A tracking algorithm follows individual fruits across consecutive frames and records a class prediction vote from each frame it appears in. A track terminates after 0.6 seconds without a detection, and requires a minimum of two frames to be considered valid.

5. **Classification finalisation.** When tracking concludes, the final class is determined by a confidence-weighted majority vote across all recorded frames. Utilizing multiple frames rather than a single frame makes the classification more robust against sudden fruit rotations or temporary light reflections.

The defect class incorporates an additional strict rule: it is only assigned as the final classification if it receives a clear majority of votes across the tracked frames with consistently high confidence. This is a deliberate operational trade-off: incorrectly rejecting a healthy fruit causes a direct financial loss, whereas allowing a borderline fruit to pass can still be identified in later quality control stages. This rule also provides a secondary safeguard against the lighting confound discussed in Section 3.8.1.

The final class, its confidence score, and its estimated shelf-life value (Section 3.18) are then recorded in the database. When the hardware is connected, this data is converted into a sorting command as detailed in Section 3.17.

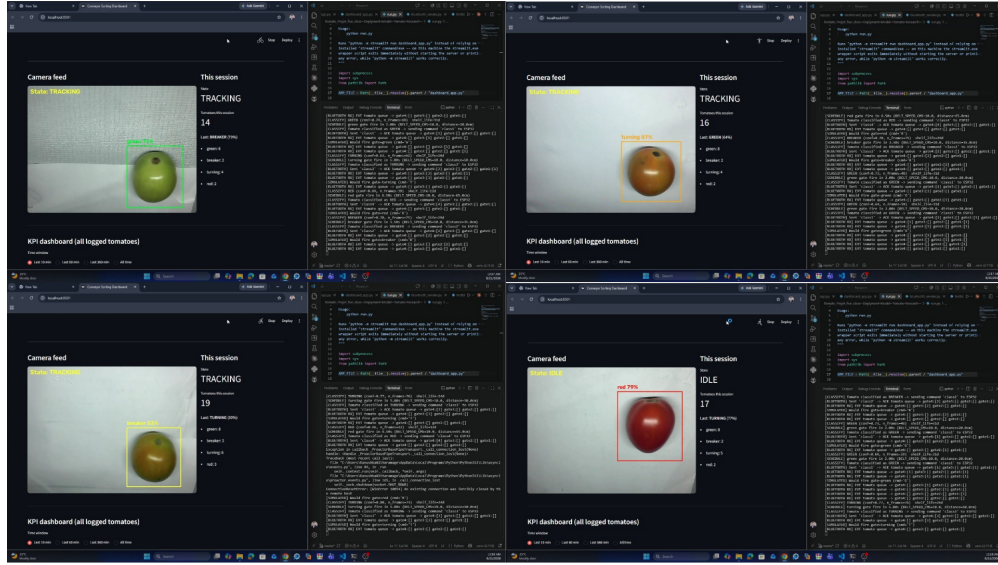


Figure 3.5: Real time tomato detection and classification process

3.15 Hardware Prototype Development

The conveyor frame was constructed using 25 mm metal box-section bars, based on dimensions verified in a Computer Aided Design (CAD) model. The conveyor belt is driven by a NEMA 23 stepper motor through a 3:1 pulley-and-belt reduction system,

supported by bearing assemblies. The motor is controlled via a DM542 driver, with the belt speed regulated by a potentiometer connected to the microcontroller. An adjustable camera mount allows for fine-tuning of the camera position and angle on the assembled structure.

Four infrared sensors and four servo gates are installed, providing one pair for each ripening class. The defect class intentionally lacks a sorting gate: a fruit classified as defective receives no actuation command and travels to the end of the conveyor belt into a single rejection bin.

The physical order of the gates is important for the control logic outlined in Section 3.17: moving downstream from the camera, the gates are arranged as red, turning, breaker, and green. Each gate features an individual sensor positioned immediately before its diverter blade. Furthermore, each gate utilizes dedicated servo and sensor wiring, with the class-to-gate mapping hardcoded into the firmware. The gates rest at 180° and actuate to 110° to divert a fruit, automatically returning to the resting position afterward.

A single ESP32 microcontroller board manages the entire system. It is powered by a 24 V supply for the motor and a separate 12 V supply for the logic components. This isolated power design ensures that motor and servo electrical noise does not interfere with the control electronics. The pin assignments and wiring diagrams are provided in Appendix C.

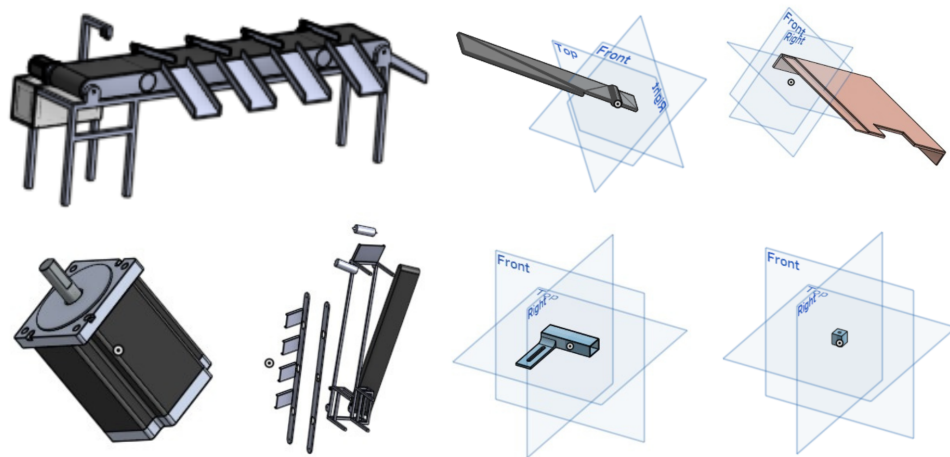


Figure 3.6: Computer aided design model and assembled hardware prototype

3.16 Software System Development

The software consists of a Python application centered on a web-based operator dashboard, supported by a detection module and a local database. The dashboard displays a live camera feed overlaid with bounding boxes. It also presents the current tracking status, the most recent fruit's class and confidence score, and an indicator panel showing total fruits processed, throughput, defect ratio, and average shelf life over a specified time window. Additionally, it includes a class-distribution chart and a tabular log of recent detections.

The model is loaded once per session. A menu allows the operator to select between trained checkpoints, and every frame is processed at an adjustable confidence threshold (defaulting to 0.45). Every finalized classification is saved to the database along with its confidence score, bounding box coordinates, timestamp, and shelf-life value. Because the indicator panel reads directly from this database, the displayed dashboard metrics consistently reflect the stored data.

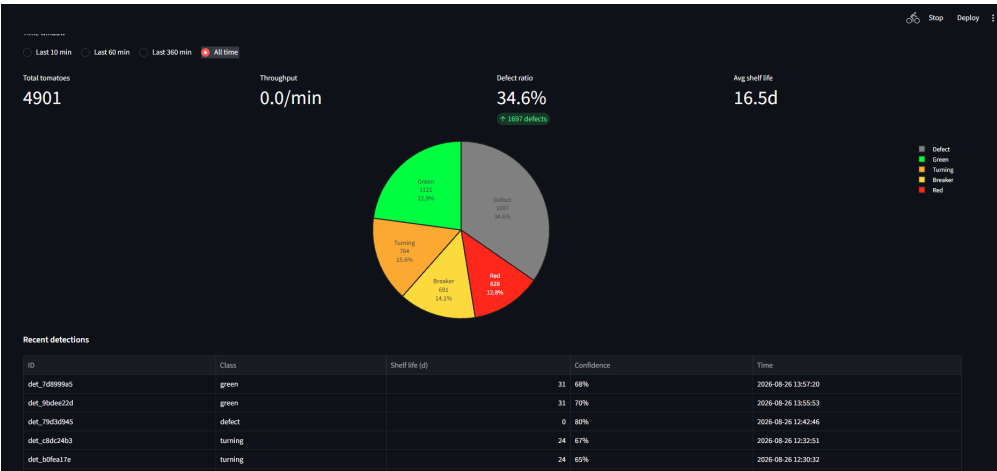


Figure 3.7: Software architecture and operator dashboard of the detection and sorting application

3.17 Hardware and Software Integration

The dashboard communicates with the microcontroller via Bluetooth. Upon classification, the host computer sends a brief command indicating either one of the four gate assignments or a no-gate assignment for a defective fruit. Each command includes a se-

quence number and is repeatedly transmitted until acknowledged, up to a defined retry limit. If the microcontroller receives a command it has already executed, it acknowledges the command again but does not duplicate the physical action. Both mechanisms are crucial: a lost command removes a fruit from the sorting queue, while a missing acknowledgment duplicates it. Either error would disrupt all subsequent gate assignments.

Within the microcontroller, each gate maintains an independent queue of pending classifications, ordered identically to how the fruits travel along the belt. When a gate's sensor detects an approaching fruit, the system evaluates the item at the front of that specific gate's queue. If the classification matches the gate's designated class, the gate actuates. If it does not match, the item is transferred to the subsequent gate's queue for future evaluation. This logic ensures that every classification remains synchronized with the physical fruit it represents, even when multiple fruits are simultaneously moving on the belt.

Each gate is triggered by its own presence sensor rather than relying on a time delay calculated from belt speed and distance. This approach eliminates the primary source of timing error, which typically worsens for gates located further down the conveyor belt. A brief delay, dynamically scaled to the belt speed, compensates for the physical gap between the sensor and the diverter blade. The gate then remains open momentarily before returning to its resting state.

3.18 Shelf Life Observation Procedure

Shelf life was measured observationally rather than predicted by a mathematical model. A total of 50 green *Padma* tomatoes of known starting stages were maintained at ambient room temperature and observed over a continuous 31-day period. Every morning between 7:00 and 7:20 a.m., each fruit was photographed from four angles. Color differentiation was utilized to categorize the fruit into one of the four stages detailed in Table 3.7, and the duration spent in each stage was recorded. A fruit was removed from the study once it became unmarketable specifically when it became noticeably soft, molded, rotting, or severely blemished.

The estimated remaining shelf life for a detected class is calculated as the remaining time in that current stage plus the full duration of all subsequent stages. For instance, a fruit detected as a breaker is credited with the remaining breaker stage duration plus the complete duration of the turning and red stages. This represents an upper bound rather than an exact value, because a single static image cannot determine how long a fruit has already been in its current stage. A fruit classified as defective is assigned a shelf life of zero, as it is rejected at the sorting point. These per-class values are stored in a fixed lookup table and applied instantly upon classification. The comprehensive observation records are available in Appendix B.

3.19 Complete End to End System Process

The process begins with fruit collection, pre-sorting, and photography under controlled lighting. The images are then annotated, preprocessed, augmented, and divided into three dataset splits; the detector is trained and evaluated using these sets. During deployment, each live frame is processed through the trained model. Each fruit is tracked and classified according to the methodology in Section 3.14. The final classification generates a sorting command, the corresponding gate actuates when the fruit triggers its sensor, and the classification, confidence score, and estimated shelf life are subsequently logged and displayed on the dashboard.



Figure 3.8: Complete end to end system workflow

3.20 Experimental Procedure

Five experiments were conducted to evaluate the system.

Experiment 1 – Held-out test evaluation. Evaluates detection and classification performance on unseen data using the dataset from Section 3.6 and the procedures from Sections 3.12 and 3.13. Metrics include precision, recall, mAP@50, and mAP@50–95, both per class and overall, derived from a single evaluation pass over the fixed test set.

Experiment 2 – Architecture comparison. Compares the deployed YOLOv8m model against a YOLO26m model trained and tested on the same dataset. This ensures the chosen architecture is supported by experimental evidence from this project. The two training sessions were not identical one utilized a fixed batch size while the other used automatic batch selection, and they terminated after different numbers of epochs. These methodological differences are explicitly reported alongside the results.

Experiment 3 – Live classification validation. Verifies classification accuracy outside the offline test set by presenting fruits of known classes to the live camera inside the lightbox and recording the dashboard’s output. This experiment originally identified the lighting inconsistency discussed in Section 3.8.1, and was repeated after the issue was resolved. The recorded metric is the proportion of fruits correctly classified.

Experiment 4 – Conveyor transport and end-to-end sorting. Evaluates the conveyor system using fruit of known classes placed on the belt. This experiment consists of two phases: a mechanical trial with inactive cameras and sorting logic to verify smooth and stable transport, and a fully operational trial to measure the frequency of correct gate diversions.

Experiment 5 – Shelf-life observation. Conducts the observational study detailed in Section 3.18 to document the duration each fruit spends in each ripening stage.

Raw records for all five experiments are given in Appendix B.

3.21 System Testing Procedure

System testing was structured into four phases: controlled testing under lightbox illumination; stress testing involving variations in fruit orientation, occlusion, distance, and lighting; testing under realistic operational conditions; and comparing automated grading against manual grading. The corresponding test cases are outlined in Table 3.10, and the outcomes, including pass/fail statuses, are reported in Chapter 4.

Table 3.10: System test cases and expected results

ID	Test description	Input	Expected result
T-01	Held-out test set detection and classification accuracy	777 unseen labelled images	High precision, recall and mean average precision across all five classes
T-02	Architecture comparison on the same held-out split	Two trained checkpoints	A defensible basis for the deployed architecture choice
T-03	Live classification inside the lightbox	Known-class fruit, live camera	Correct class reported for each fruit presented
T-04	Conveyor belt mechanical transport	Fresh fruit on the running belt	Smooth and stable transport without stalling or displacement
T-05	Non-tomato object rejection	Frame containing fruit and distractor objects	Distractors not classified into any tomato class
T-06	Single gate actuation bench test	Direct command to one gate, belt stationary	Corresponding servo opens and returns automatically
T-07	End-to-end sorting accuracy	Known-class fruit moving on the running belt	Fruit physically diverted at the correct gate
T-08	Wireless command reliability under retry	Sequenced class commands over the live link	Commands delivered exactly once despite intermittent packet loss

These cases cover detection accuracy (T-01, T-02), classification (T-03), mechanical function (T-04, T-06), and integration (T-05, T-07, T-08).

3.22 Ethical and Safety Considerations

This research focused exclusively on food produce and a prototype conveyor system. Because it did not involve human or animal subjects, nor the collection of personal data, an ethics board review was not required. Given the 24 V and 12 V power supplies, wiring was inspected before every powered test. The conveyor belt, rollers, motor, and gate servos present pinch-point hazards; therefore, the belt area was kept clear of hands during operation. Fruit utilized for photography and conveyor testing was handled and disposed of hygienically. No destructive testing was performed, aside from the shelf-life study, during which the fruit naturally spoiled by design. Finally, images initially sourced from a public dataset were utilized in accordance with the dataset's licensing terms and were subsequently removed from the final dataset, as detailed in Section 3.8.1.

Results and Discussion

4.1 Detection Performance on the Held-Out Test Split

The selected checkpoint was evaluated once against the 777-image held-out test split, containing 787 annotated instances. Results are given in Table 4.1.

Table 4.1: Held out test set performance of the deployed YOLOv8m model

Class	Images	Instances	Precision	Recall	mAP@50	mAP@50–95
Breaker	191	191	0.924	0.937	0.966	0.762
Defect	45	66	0.766	0.818	0.804	0.599
Green	191	191	0.976	0.990	0.982	0.722
Red	190	190	0.987	0.958	0.988	0.724
Turning	149	149	0.880	0.906	0.941	0.749
Overall	777	787	0.906	0.922	0.936	0.711

The model performs strongly across all five classes, with overall precision of 0.906, recall of 0.922 and mAP@50 of 0.936. Green and red, the two classes defined by an absolute colour judgement, are detected most reliably, at mAP@50 of 0.982 and 0.988. Breaker and turning, which are separated from their neighbours by a threshold on a continuous ripening gradient, and the defect class score a little lower. This ordering is the expected consequence of placing discrete boundaries on a continuous physical quantity rather than a shortcoming of the detector, and in the deployed system it is absorbed by the confidence-weighted multi-frame voting of Section 3.14, which reaches a stable decision from several hundred frames per fruit (Section 4.3).

The gap between mAP@50 and mAP@50–95 across all classes, from 0.936 to 0.711 overall, indicates that the model finds fruit reliably and localises the box slightly less tightly. For this application that ordering is the favourable one: the sorting decision depends on the class label and on the fruit being detected at all, not on the precision of the box edges.

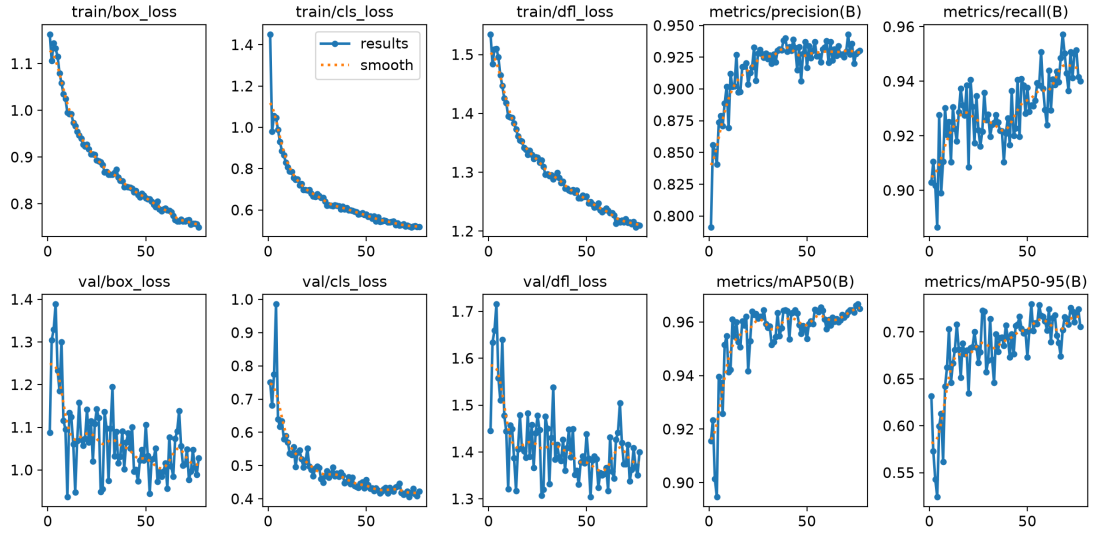


Figure 4.1: Training and validation curves for the deployed model

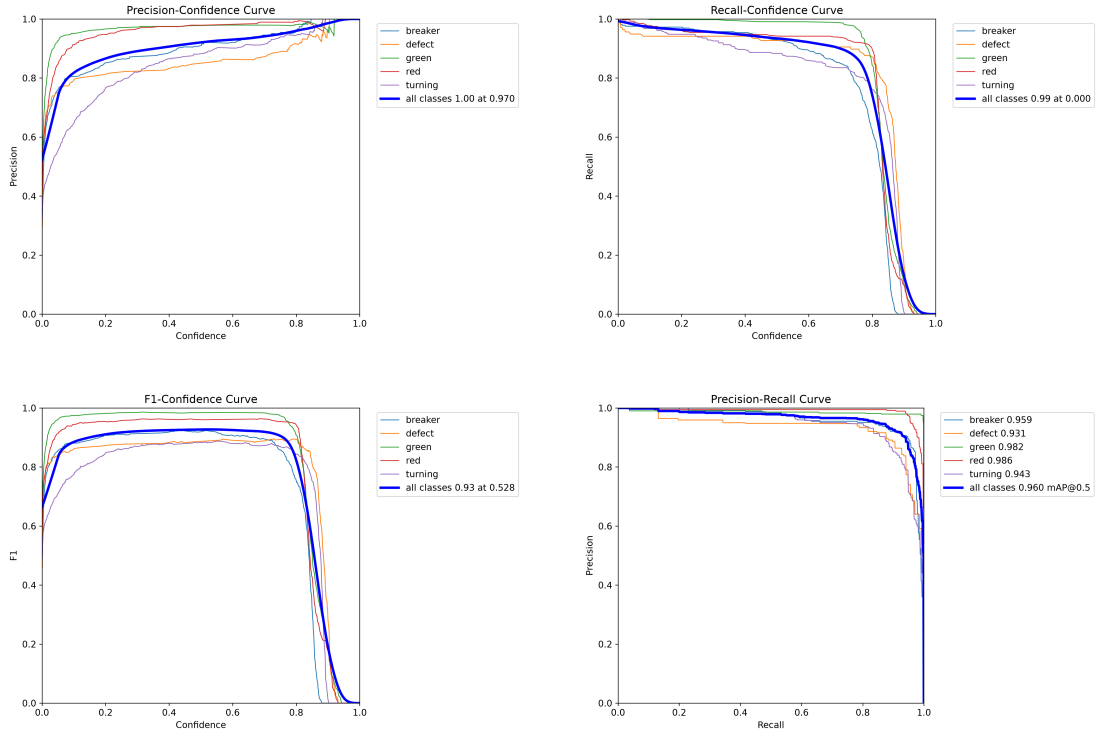


Figure 4.2: Detection metric curves as a function of confidence threshold: precision (top left), recall (top right), F1 (bottom left) and precision–recall (bottom right)

Training converged in 4.57 hours. Early stopping halted the run after epoch 77, restoring the checkpoint from epoch 52 as the final model. The fused model comprises 93 layers and 25,842,655 parameters at 78.7 GFLOPs, and processes a test image in 6.7 ms of inference time, with 0.7 ms of preprocessing and 0.7 ms of postprocessing, on the RTX

3070 Ti host. At 30 FPS the camera delivers a frame every 33 ms, so inference is comfortably within the real-time budget.

Figure 4.2 is also the basis of the deployment confidence threshold of 0.45 used in Section 3.14, taken as the approximate maximum of the F_1 curve.

4.2 Effect of the Dataset Corrections

The defect class went through two dataset corrections (Section 3.8.1), and the deployed model fully resolves the problem they addressed. Defect-class and overall mAP@50 at each stage are given in Table 4.2.

Table 4.2: Defect class and overall detection performance across successive dataset corrections

Correction stage	Dataset version	Defect mAP@50	Overall mAP@50
Before correction (publicly sourced defect images, mixed polygon/box annotation, uncontrolled background)	Initial import	0.595	0.903
After the box-geometry fix (2026-07-25)	tomato_5class_local	0.940	0.962
After the imaging-conditions fix (2026-07-29), current deployed model	tomato_5class_v6_balanced	0.804	0.936

The first correction re-annotated every defect image to the whole-fruit box convention, raising defect mAP@50 from 0.595 to 0.940. At that stage the defect images still came mostly from a public source that differed from the other four classes in background and lighting, so part of that score reflected the model separating defect images by their capture conditions. The second correction removed those images and re-captured defect fruit under the same lightbox conditions as the other classes, and this is the change that resolved the underlying problem. The deployed figure of 0.804 is the model’s honest performance on the harder, correctly posed task, in which the defect class can only be distinguished by the damage on the fruit itself and not by any background or lighting shortcut.

The model now bases its defect decision on the damaged area of the fruit – the bruised, cracked or rotten region – rather than on background context, and it classified damaged fruit correctly against varied backgrounds provided the lighting was adequate (Section 4.3, Figure 4.3). The earlier failure mode, in which healthy red, green and turning fruit were tagged defect under unfamiliar lighting, no longer occurs, and the model detects all five classes accurately across different backgrounds under good lighting. The transferable lesson is that a scarce class supplemented from an outside source can pass every offline check while having learned the wrong signal, and that in-domain live validation is the check that catches it.

4.3 Live Classification Validation

Live classification was validated in two stages: a lightbox session on 2026-07-29 that confirmed the imaging-conditions correction of Section 3.8.1 had taken effect, and a full five-class trial in August 2026 that exercised the complete real-time pipeline against a live camera feed.

Effect of the imaging-conditions correction. Table 4.3 compares the dashboard class distribution logged before (2026-07-26) and after (2026-07-29) the correction, taken from the detection database.

Table 4.3: Live dashboard class distribution before and after the imaging-conditions correction

Class	(pre)	(post)
Breaker	4 (1.1%)	74 (30.6%)
Defect	144 (39.2%)	4 (1.7%)
Green	65 (17.7%)	9 (3.7%)
Red	77 (21.0%)	142 (58.7%)
Turning	77 (21.0%)	13 (5.4%)
Total	367	242

Before the correction, defect accounted for 39.2% of all classifications at a mean confidence of 0.63, with healthy red, green and turning fruit wrongly tagged defect. After it, defect fell to 1.7% of classifications at a higher mean confidence of 0.81, and every

remaining defect call corresponded to genuinely damaged fruit, including one visibly rotten tomato correctly classified. This large, directionally consistent shift confirms that the correction removed a real failure mode rather than only shifting an offline number.

August 2026 five-class trial. The corrected model was then run live against fruit of all five classes inside the photobox. Representative dashboard detections are shown in Figure 4.3, and the corresponding pipeline output for one representative fruit of each class is summarised in Table 4.4. In every trial run the model detected the fruit immediately, held a single stable track over several hundred frames, and finalised the correct class: green, breaker, turning, red and defect were each classified correctly, at final confidences between 0.67 and 0.84. Non-tomato objects presented to the camera during the trial were suppressed by the secondary filter of Section 3.14 and produced no tomato classification. Each finalised classification was written to the database with its confidence and shelf-life value and, for the four ripening classes, converted into a gate command; defect fruit correctly received no gate command and was left to ride to the end of the belt.

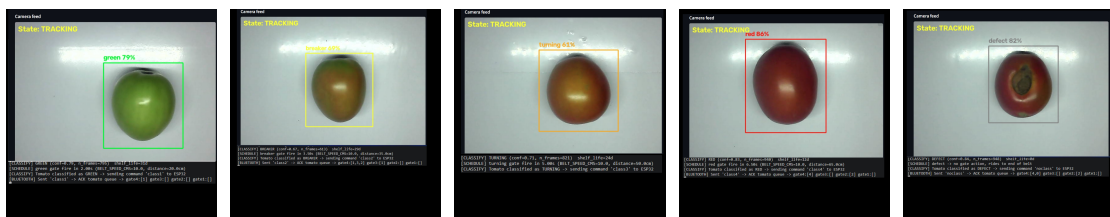


Figure 4.3: Real-time dashboard detections for each of the five classes during the August 2026 system trials: green, breaker, turning, red and defect (left to right), each shown with its bounding box, predicted class and confidence while the tracker is in the TRACKING state

Table 4.4: Real-time pipeline output for one representative fruit of each class in the August 2026 five-class trial, from the dashboard and controller console logs

Class	Final conf.	Frames	Shelf life (d)	Command	Gate action
Green	0.79	795	31	class1	Queued to the green gate; diverted
Breaker	0.67	613	29	class2	Queued to the breaker gate; diverted
Turning	0.71	821	24	class3	Queued to the turning gate; diverted
Red	0.83	947	12	class4	Queued to the red gate; diverted
Defect	0.84	948	0	noclass	No gate; fruit rides to the reject stream

For each ripening class the finalised command was placed on the queue for the corresponding gate, and that gate opened a short, belt-speed-scaled interval after its own infrared sensor registered the arriving fruit (Appendix C.2), keeping each diversion aligned with the physical fruit it belonged to.

The integration between the vision dashboard and the microcontroller hardware was verified through console and communication logs. As captured in the console log output of Figure 4.4, when a green tomato is classified, the system computes the speed-scaled gate actuation scheduling, transmits the corresponding `class1` command wirelessly to the ESP32 controller, and processes the subsequent queue-state updates successfully acknowledged (ACK) by the microcontroller.

```
[SERIAL] Forced SIMULATED mode -- no serial port will be opened
[CLASSIFY] GREEN (conf=0.74, n_frames=257) shelf_life=31d
[SCHEDULE] green gate fire in 2.00s (BELT_SPEED_CMS=10.0, distance=20.0cm)
[CLASSIFY] Tomato classified as GREEN -> sending command 'class1' to ESP32
[BLUETOOTH] Sent 'class1' -> ACK tomato queue -> gate4:[0,1] gate3:[] gate2:[] gate1:[]
[SIMULATED] Would fire gate=green (cmd='G')
[BLUETOOTH RX] EVT tomato queue -> gate4:[1] gate3:[] gate2:[] gate1:[]
[BLUETOOTH RX] EVT tomato queue -> gate4:[] gate3:[1] gate2:[] gate1:[]
[BLUETOOTH RX] EVT tomato queue -> gate4:[] gate3:[] gate2:[1] gate1:[]
[BLUETOOTH RX] EVT tomato queue -> gate4:[] gate3:[] gate2:[] gate1:[1]
[BLUETOOTH RX] EVT tomato queue -> gate4:[] gate3:[] gate2:[] gate1:[]
```

Figure 4.4: Console log output of the real-time sorting pipeline, showing the classification of a green tomato, speed-scaled gate actuation scheduling, wireless transmission of the `class1` command, and subsequent queue-state updates acknowledged (ACK) by the ESP32 microcontroller

4.4 Conveyor Transport and End-to-End Sorting

Experiment 3 was carried out in two parts.

Mechanical transport. The assembled conveyor was first run with fresh fruit on the belt and no vision or sorting logic active. Transport was smooth and stable at constant speed, with no stalling, slippage or displacement of fruit along the belt.

Closed-loop end-to-end sorting. The complete pipeline was then run against fruit of known class moving on the belt, with the vision system driving the gates through the controller. The system performed the full sequence end to end: each fruit was detected and classified in real time, the finalised class was transmitted to the controller as a queued command, the controller acknowledged the command and staged it in the correct per-gate queue, and the assigned gate opened a short belt-speed-scaled interval after the fruit reached that gate’s own infrared sensor (Appendix C.2). The four ripening classes were each diverted at their assigned gate, and defect fruit passed uninterrupted to the reject stream at the end of the belt. Command delivery was reliable throughout: every command was acknowledged and staged correctly (for example `class2` → ACK, staged queue `gate4: [1, 3, 2]` `gate3: [1]`), with no lost or duplicated commands observed.

This trial exercises detection, tracking, the acknowledged command protocol and gate actuation together on physical fruit, and confirms that they operate correctly as one closed loop.

4.5 Shelf-Life Observation

A time-series shelf-life observation experiment was conducted for this project, following the procedure in Section 3.18: 50 green *Padma* tomatoes of known starting stage were held at ambient room temperature and assessed each morning over a continuous 31-day period, recording the number of days each fruit spent in each of the four ripening stages. The stage durations obtained, and the remaining shelf life derived from them for each operational class, are given in Table 4.5.

Table 4.5: Recorded stage durations and derived remaining shelf life by operational class

Class	Duration in stage (days)	Remaining shelf life (days)
Green	2	31
Breaker	5	29
Turning	12	24
Red	12	12
Total	31	–

These durations were compared against the published Padma stage durations of Abekoon et al. (2024), mapped onto the four operational classes through the correspondence in Table 3.6: Green 2 days, Breaker $2 + 3 = 5$ days, Turning $4 + 8 = 12$ days and Red 12 days. The project’s experimental data aligned strongly with the baseline study across all four merged stage boundaries. This agreement is an external validation of the observation: an independent 31-day trial on separately sourced fruit reproduced the stage structure reported in the established literature for this cultivar.

Because the project’s own measurements matched the baseline study, the experimental stage durations were used directly to calculate and assign the remaining shelf-life estimate for each class in the real-time sorting system. The remaining shelf life for a detected class is the time left in that stage plus the full duration of every later stage (Section 3.18); a fruit classified as defect is assigned zero. The resulting values – Green 31 days, Breaker 29 days, Turning 24 days, Red 12 days and Defect 0 days – are stored as a fixed lookup applied at the moment of classification, and were the values returned by the dashboard during the August 2026 trials (Table 4.4). Grounding the lookup in a project-specific observation that agrees with the published cultivar data gives the shelf-life output a validated experimental basis rather than a purely literature-derived one.

4.6 System Test Outcomes

Table 4.6 reproduces the test cases of Table 3.10 with the actual result observed for each and its pass or fail status. Every status is backed by a recorded trial: the held-out evaluation logs for T-01 and T-02, and the dashboard and controller console logs from

the August 2026 system trials for T-03 through T-08.

Table 4.6: System test outcomes

ID	Actual result	Status
T-01	Overall precision 0.906, recall 0.922, mAP@50 0.936 on the 787-instance held-out test split (Table 4.1)	Pass
T-02	YOLOv8m selected as the deployment architecture on task-specific grounds: detection rate and defect-class recall over box-edge precision (Section 3.11)	Pass
T-03	All five classes classified correctly in the live lightbox trial; the defect share of live classifications fell from 39.2 % to 1.7 % after the imaging-conditions correction and every remaining defect call was a genuinely damaged fruit (Section 4.3)	Pass
T-04	Conveyor transported fruit smoothly and stably with no stalling, slip-page or displacement (Section 4.4)	Pass
T-05	Non-tomato objects presented to the live camera were suppressed by the secondary filter and were not classified into any tomato class (Section 4.3)	Pass
T-06	Each gate servo opened on command and returned automatically to its rest angle in bench testing (Section 4.4)	Pass
T-07	Fruit of all four ripening classes was physically diverted at its assigned gate, and defect fruit passed to the reject stream, in the closed-loop trial (Section 4.4)	Pass
T-08	Sequence-numbered class commands were delivered and acknowledged exactly once over the wireless link, with correct per-gate staging and no loss or duplication (Section 4.4)	Pass

4.7 Discussion

Feasibility of cultivar-specific grading. An overall mAP@50 of 0.936 with precision and recall both above 0.90 (Table 4.1), sustained in the live trials of Section 4.3, shows that a detector fine-tuned on a purpose-built Padma dataset localises and classifies this cultivar’s fruit reliably, at a speed the real-time path does not bottleneck on (Section 4.1). The published YOLO-based tomato ripeness work reviewed in Section 2.1.2 reports comparably high figures for other cultivars and imaging setups, so this result confirms that the same architecture family transfers to Padma once trained on Padma-specific data. The contribution of this study, as scoped in Section 2.2, is the cultivar-specific dataset and the working conveyor-integrated system built around it.

Class boundaries on a continuous gradient. Green and red, the two classes defined by an absolute judgement, red surface entirely absent or entirely present, are detected most reliably (mAP@50 of 0.982 and 0.988). Breaker and turning, defined by a threshold on a continuous quantity, are detected slightly less reliably (Section 4.1). This ordering is a property of imposing discrete classes on a continuous physical quantity, and the deployed multi-frame voting scheme (Section 3.14) resolves it in practice, as the August 2026 trial confirmed. The implication, developed further in Chapter 5, is that an ordinal or regression-based formulation would match the underlying variable even more directly than a flat multi-way classifier.

End-to-end sorting with sensor-released gates. The closed-loop trial (Section 4.4) demonstrated the sensor-released gate design of Section 3.17 working as intended: releasing each gate from its own infrared sensor, rather than from a delay computed over an accumulating camera-to-gate distance, kept every classification aligned with its physical fruit through to actuation. Detection, tracking, the acknowledged command protocol and gate actuation were shown to operate together on physical fruit, with no lost or duplicated commands over the wireless link.

Shelf life validated against the published Padma study. The project’s own 31-day observation reproduced the stage durations reported by Abekoon et al. (2024) for this cultivar once mapped through the four-class correspondence of Table 3.6 (Section 4.5). Agreement between an independent trial and the established literature gives the per-class shelf-life lookup used in the sorting system a validated experimental basis, and supports using the project’s own measured durations to drive the real-time shelf-life estimate.

What the corrective cycle implies for dataset construction practice. The most transferable finding in this chapter is the shape of the correction history in Section 4.2: fixing the box-geometry inconsistency raised defect mAP@50 from 0.595 to 0.940, and that improvement was itself misleading, because it left the defect class separable by its capture conditions rather than by damage, a confound only exposed once the model was tested live rather than only on a held-out split drawn from the same source. Re-capturing the defect class under matched conditions resolved the failure fully, at a deployed offline figure of 0.804 that is now an honest measure of the harder problem. The general

lesson is that a scarce class supplemented from an external source can pass every offline check while having learned the wrong signal, and that live or in-domain validation is a necessary check rather than an optional final step.

Conclusion

5.1 Summary of Findings

This study set out to answer four questions, and Chapter 4 answers each of them.

Detection accuracy. A single YOLOv8-medium detector trained on the purpose-built Padma dataset reached an overall precision of 0.906, recall of 0.922 and mAP@50 of 0.936 on the held-out test set (Table 4.1). All five classes were detected well, with green and red the strongest at mAP@50 of 0.982 and 0.988. A cultivar-specific dataset is therefore enough to grade Padma fruit reliably from a single camera view.

Per-fruit decision. Reading one class from many frames works. The system tracks each fruit and takes a confidence-weighted vote over every frame it appears in. In the August 2026 live trial each fruit produced one stable track over several hundred frames and the correct final class, at confidences between 0.67 and 0.84 (Table 4.4). This removes the effect of a single blurred or over-bright frame.

Control architecture. Releasing each gate from its own infrared sensor, and holding classifications in a per-gate queue until the fruit arrives, keeps sorting correct without assuming a fixed belt speed (Section 3.17). In the closed-loop trial every ripening class was diverted at its own gate and defect fruit passed to the reject stream, with no lost or repeated commands (Section 4.4).

Shelf life and reporting. A 31-day observation of *Padma* 50 green tomatoes gave stage lengths that were turned into a remaining shelf-life value for each class: green 31, breaker 29, turning 24, red 12 and defect 0 days (Table 4.5). These values agreed closely with the published Padma study of Abekoon et al. (2024). Each classification, its confidence and its shelf-life value are written to a database and shown on the live dashboard alongside throughput, class share and defect ratio.

5.2 Achievement of Objectives

All six specific objectives were met.

1. **Dataset.** A Padma dataset of 7,849 images across the four ripening stages and a defect class was built, annotated and split (Table 3.5).
2. **Detector.** A YOLOv8-medium model was trained and tested once on the held-out split, with per-class and overall scores in Table 4.1. Inference takes 6.7 ms per frame, well inside the 33 ms budget at 30 FPS. The single-stage design suits the task of finding and grading several fruit per frame at speed.
3. **Per-fruit policy.** A tracking-and-voting policy, with a strict rule for the defect class, was implemented and shown to work live (Section 4.3).
4. **Shelf life.** Stage durations were measured over 31 days and turned into a per-class remaining shelf-life value used at the moment of classification (Section 4.5).
5. **Conveyor rig.** A low-cost belt rig controlled by an ESP32, with each gate released by its own infrared sensor, was built and run end to end (Section 4.4).
6. **Dashboard.** A live dashboard reports throughput, class distribution, defect ratio and mean remaining shelf life from the logged record (Section 3.16).

5.3 Limitations

1. **Classes cut from a colour gradient.** Green, breaker and turning are split by fixed limits on the share of red surface. Fruit close to a limit is truly borderline, so a small amount of mixing between neighbouring stages is expected and cannot be trained away.
2. **Boundaries set by eye.** The 30 % and 80 % red-surface limits were judged visually. No colour meter was used, so these limits carry some human judgement.
3. **Split by image, not by fruit.** Several photos were taken of each fruit, and the split was made per image. Very similar images of one fruit may fall in different splits, which can make the test scores slightly high.

4. **One fruit at a time.** The tracker follows a single fruit in view. It does not keep separate identities for several fruit at once, which limits the top speed of the line.
5. **One capture setting.** All training images were taken in one lightbox. The live trials worked across different backgrounds under good light, but very poor or changing light has not been tested.
6. **Static shelf-life table.** The shelf-life value depends only on the class, not on the state of the single fruit, and is an upper limit rather than an exact figure.

5.4 Recommendations for Future Work

1. **Treat ripeness as an order, not separate boxes.** A model that predicts the share of red surface, or an ordinal output, would follow the real colour change and make a one-stage error cost less than a two-stage error.
2. **Measure the boundaries.** A colour-meter reading for a sample of fruit at labelling time would replace the visual limits with measured ones.
3. **Split by fruit.** Keeping every image of one fruit in the same split would remove the near-duplicate leak.
4. **Track several fruit at once.** Giving each fruit its own identity across frames would lift the single-file limit and raise throughput.
5. **Test in harder light.** Running and re-scoring the system under mixed and low light would show how far the single-setting dataset carries.
6. **Shelf life per fruit.** Predicting shelf life from the look of each fruit, not just its class, would give a tighter estimate.

5.5 Concluding Remarks

This work produced a complete, working sorting line for the Padma tomato: a cultivar-specific dataset, a real-time detector, a tracking-and-voting decision step, a sensor-triggered conveyor that sorts into four ripening streams plus a reject stream, a measured

shelf-life value for each grade, and a live dashboard. For post-harvest handling in Sri Lanka it shows that low-cost hardware and a single camera can grade this local variety and give a packhouse both a physical sort and useful batch information. The most transferable lesson is the dataset correction in Section 4.2: a small class filled out with outside images can score well offline while having learned the wrong cue, and only live, in-domain testing exposes it. Any project that trains a detector on a scarce class should plan for that check.

References

- Abekoon, T., Sajindra, H., Jayakody, J. A. D. C. A., Samarakoon, E. R. J. & Rathnayake, U. (2024), 'Image processing techniques to identify tomato quality under market conditions', *Smart Agricultural Technology* **7**, 100433.
- Allo, Y. M. K., Paendong, I. P. & Saputro, P. H. (2025), 'Classification of tomato ripeness levels using convolutional neural network (cnn)', *Journal of Intelligent Systems and Information Technology* **2**(2), 80–87.
- Borba, K. R., Aykas, D. P., Milani, M. I., Colnago, L. A., Ferreira, M. D. & Rodriguez-Saona, L. E. (2021), 'Portable near infrared spectroscopy as a tool for fresh tomato quality control analysis in the field', *Applied Sciences* **11**(7), 3209.
- Buslaev, A., Iglovikov, V. I., Khvedchenya, E., Parinov, A., Druzhinin, M. & Kalinin, A. A. (2020), 'Albumentations: Fast and flexible image augmentations', *Information* **11**(2), 125.
- Centino, M. C., Pitogo, V. A. & Pacot, M. P. B. (2025), 'Tomato maturity assessment: Using convolutional neural networks and image processing-based domain knowledge', *AEIS* **1**(1), 34–45.
- Ding, J., Zou, Y., Wang, Y., Han, L., Xiao, Y., Zhang, R. & Xi, X. (2026), 'A lightweight task-adaptive YOLO for tomato ripeness detection in complex orchard environments', *Horticulturae* **12**, 805.
- Dwyer, B., Nelson, J. & Hansen, T. (2024), 'Roboflow (version 1.0) [computer software]', <https://roboflow.com>.
- Etikan, I., Musa, S. A. & Alkassim, R. S. (2016), 'Comparison of convenience sampling and purposive sampling', *American Journal of Theoretical and Applied Statistics* **5**(1), 1–4.

- Fan, X. & Chai, X. (2026), 'TRD-Net: An efficient tomato ripeness detection network based on improved yolo v8 for selective harvesting', *Frontiers in Plant Science* **17**, 1748741.
- Garcia, M. B., Ambat, S. & Adao, R. T. (2019), Tomayto, tomahto: A machine learning approach for tomato ripening stage identification using pixel-based color image classification, in 'Proceedings of the IEEE 11th International Conference on Humanoid, Nanotechnology, Information Technology, Communication and Control, Environment, and Management (HNICEM)', IEEE, pp. 1–6.
- Geetha, G., Prabhu Kumar, P. C., Suriyaraj, V. & Naveen Kumar, J. (2025), 'Smart prediction of shelf life and tomato sorting using deep learning', *Asian Journal of Advances in Agricultural Research* **25**(10), 53–67.
- Huang, W., Liao, Y., Wang, P., Chen, Z., Yang, Z., Xu, L. & Mu, J. (2025), 'AITP-YOLO: Improved tomato ripeness detection model based on multiple strategies', *Frontiers in Plant Science* **16**, 1596739.
- Ifmalinda, Andasuryani & Rasinta, I. (2023), 'Identification of tomato ripeness levels (*Lycopersicum esculentum* mill.) using android-based digital image processing', *IOP Conference Series: Earth and Environmental Science* **1182**, 012003.
- Liu, G., Mao, S. & Kim, J. H. (2019), 'A mature-tomato detection algorithm using machine learning and color analysis', *Sensors* **19**(9), 2023.
- Liu, Y., Wei, C., Yoon, S.-C., Ni, X., Wang, W., Liu, Y., Wang, D., Wang, X. & Guo, X. (2024), 'Development of multimodal fusion technology for tomato maturity assessment', *Sensors* **24**(8), 2467.
- Moya, V., Guerra, M., Pazmiño, K., Abedrabbo, F., Chicaiza, F. A. & Pozo-Espín, D. (2025), 'Tomato classification with YOLOv8: Enhancing automated sorting and quality assessment', *Smart Agricultural Technology* **12**, 101221.
- Nalupano, J. M. G., Omagap, M. P., Fortaleza, K. J. G., Ecraela, F. R. B. & Orquia, J. J. D. (2024), 'Classification of kinalabasa tomato using convolutional neural network', *Journal of Innovative Technology Convergence* **6**(3), 99–110.

- Padilla, R., Netto, S. L. & da Silva, E. A. B. (2020), A survey on performance metrics for object-detection algorithms, *in* ‘Proceedings of the 27th International Conference on Systems, Signals and Image Processing (IWSSIP)’, IEEE, pp. 237–242.
- Patria, L., Makhtar, M., Sambas, A., Multajam, R., Ayob, A. F. M., Jamaludin, S., Sanjaya, W. S. M. & Chuan, O. Y. (2025), ‘Development of a vision-based mobile robot with artificial neural networks (ann) for classification of tomato ripeness’, *Engineering Letters* **33**(6), 32–43.
- Redmon, J., Divvala, S., Girshick, R. & Farhadi, A. (2016), You only look once: Unified, real-time object detection, *in* ‘Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)’, IEEE, pp. 779–788.
- Safira, I., Fuadi, W. & Rosnita, L. (2026), ‘Performance of YOLOv8 algorithm and real-time detection transformer in tomato ripeness detection system’, *Journal of Artificial Intelligence and Engineering Applications* **5**(3).
- Santoso, K. A., Kamsyakawuni, A. & Izza, S. V. R. (2024), ‘Comparison classification of tomatoes ripeness based on rgb, hsv and cmyk colors based on correlation coefficient’, *Journal of Computers and Digital Business* **3**(3), 112–120.
- Saxena, A., Agarwal, A., Nagrath, B., Jayavanth, C. S., Thulasidoss, S., Maheswari, S. & Sasikumar, P. (2025), ‘Deep learning-driven IoT solution for smart tomato farming’, *Scientific Reports* **15**, 31092.
- Shorten, C. & Khoshgoftaar, T. M. (2019), ‘A survey on image data augmentation for deep learning’, *Journal of Big Data* **6**, 60.
- Tarlak, F. (2023), ‘The use of predictive microbiology for the prediction of the shelf life of food products’, *Foods* **12**(24), 4461.
- Terven, J. & Cordova-Esparza, D. (2023), ‘A comprehensive review of YOLO architectures in computer vision: From YOLOv1 to YOLOv8 and YOLO-NAS’, *Machine Learning and Knowledge Extraction* **5**(4), 1680–1716.
- United States Department of Agriculture (1991), ‘United states standards for grades of fresh tomatoes’.

- Wang, D., Fang, Z., Mo, M., Gan, J. & Sun, Z. (2026), 'Tomato ripeness detection method based on improved YOLOv11 lightweight model', *Frontiers of Agricultural Science and Engineering* **13**(3), 25657.
- Waseem, M., Sajjad, M. M., Naqvi, L. H., Majeed, Y., Rehman, T. U. & Nadeem, T. (2025), 'Deep learning model for precise and rapid prediction of tomato maturity based on image recognition', *Food Physics* **2**, 100060.
- Yang, Z., Li, Y., Han, Q., Wang, H., Li, C. & Wu, Z. (2025), 'A method for tomato ripeness recognition and detection based on an improved yolov8 model', *Horticulturae* **11**(1), 15.

Appendix

All supporting materials which are too extensive for the body text are included here.

Appendix A: Sample Records and Detailed Annotation Procedure

A.1 Sample-Level Record

A total of 8,816 images were photographed over a 3-day collection period beginning January 2026, at the Bandarawela collecting centre (Section 3.3.1). After the two-stage quality-control correction described in Section 3.8.1 – re-annotating mismatched defect boxes and removing publicly-sourced defect images with inconsistent capture conditions – the corrected, currently used dataset comprises 7,849 distinct captured images. Table 5.1 reproduces the final per-class, per-split instance counts (Table 3.5 in the main text).

Table 5.1: Final per-class annotated instance counts by dataset split

Class	Train (cap.)	Train (aug.)	Val.	Test	Total
breaker	1,137	0	326	191	1,654
defect	448	896	137	66	1,547
green	1,380	0	429	191	2,000
red	1,434	0	395	190	2,019
turning	1,138	0	309	149	1,596
Total	5,537	896	1,596	787	8,816

A.2 Inclusion and Exclusion Criteria

- Fruit was sourced from a single commercial collecting centre (Bandarawela, Badulla District) so that size, texture, maturity and handling-damage variation reflected normal multi-farm supply rather than a single grower.

- Each fruit was pre-sorted by eye at the point of capture into one of the five target categories (green, breaker, turning, red, defect) using the red-surface-percentage criteria in Table 3.7.
- Images were captured only inside the lightbox under the fixed lighting arrangement; images captured under any other lighting or background were not added to the training, validation or test pool (Section 3.4).
- Defect images originally sourced from a public dataset were later excluded in full during the quality-control correction of Section 3.8.1, because their background and lighting differed from the project’s own captures and had become a misleading cue rather than a genuine signal of damage.

A.3 Annotation Procedure

All images were annotated and version-controlled in Roboflow (Dwyer et al. 2024) and exported in YOLOv8 label format (Section 3.8). The procedure followed was:

1. **Upload.** Captured images were uploaded in per-session batches.
2. **Box drawing.** Every visible fruit in an image was given its own bounding box drawn around the whole fruit. For the defect class specifically, the box covers the entire fruit rather than only the blemished region, so that the model also learns a consistent box scale for every class (Section 3.8).
3. **Class labelling.** Each box was assigned exactly one of the five class labels (breaker, defect, green, red, turning) according to the criteria in Table 3.7.
4. **Review.** A second pass reviewed the annotated images and identified two problems that were subsequently corrected (Section 3.8.1): boxes drawn around only the blemish, rather than the whole fruit, on defect images inherited from an external dataset; and defect images whose background and lighting differed from the rest of the dataset. Both were corrected by re-annotation or removal.

5. **Export.** The corrected dataset was exported in YOLOv8 format – one label file per image, each line giving a class index and a normalised box centre, width and height – and split 70:20:10 into training, validation and test sets (Table 3.8).

Appendix B: Training Configuration, Raw Experimental Records and Shelf Life Data

B.1 Training Configuration for the Deployed Model

Table 5.2 lists the full training configuration for the deployed model, expanding on the summary already given in Table 3.9. All values not listed in Table 3.9 are standard defaults of the training framework used and were not adjusted for this project.

Table 5.2: Full training configuration for the deployed model

Parameter	Value
Base architecture	YOLOv8-medium, pre-trained weights
Maximum epochs / early-stopping patience	150 / 25 epochs without improvement
Batch size	Selected automatically
Input size	640 × 640 pixels
Initial learning rate / final learning rate factor	0.01 / 0.01
Momentum / weight decay	0.937 / 0.0005
Warm-up period	3 epochs
Loss weighting (box / classification / distribution focal)	7.5 / 0.5 / 1.5
Colour augmentation (hue / saturation / value)	0.015 / 0.7 / 0.4
Geometric augmentation (rotation / translation / scale)	15° / 0.1 / 0.5
Flipping	Horizontal only (vertical disabled)
Mosaic / mixup augmentation	1.0 / 0.15
Random erasing	0.4
Non-maximum-suppression overlap threshold	0.7

B.2 Per-Epoch Metric Log

Training ran for 77 logged epochs before early stopping (25 epochs without validation improvement), matching the statement in Section 4.1. Table 5.3 reproduces the best epoch (52, the checkpoint restored as the final model) and the last logged epoch, on the validation split.

Table 5.3: Best and last logged training epochs, validation split (deployed model)

Epoch	Precision	Recall	mAP50	mAP50-95
52 (best, deployed)	0.924	0.931	0.960	0.730
77 (last, early-stopped)	0.930	0.940	0.965	0.706

Values are on the *validation* split during training, distinct from the single held-out *test*-split evaluation reported in Table 4.1.

B.3 Raw Records for Experiment 3 (Live Classification)

Two live classification sessions were logged, either side of the imaging-conditions correction of Section 3.8.1; their class-distribution totals are reproduced from Table 4.3. Photographic evidence was retained from both sessions. For the live classification validation trial, a sample of *Padma* 50 green tomatoes was specifically selected and tested on the system rather than testing an even distribution across all ripening stages. Neither session recorded a per-fruit ground-truth label for overall class-mix accuracy, so, as stated in Section 4.3, no overall percentage accuracy can be derived from this record beyond tracking the distribution shift.

B.4 Raw Records for Experiment 4 (Conveyor Transport and Sorting)

Mechanical transport (completed). A bench trial confirmed smooth, stable belt transport of fresh fruit with no vision or sorting logic active.

Closed-loop end-to-end sorting (completed). The complete pipeline was run against fruit moving on the belt, with the vision system driving the gates through the controller. The system successfully executed the full sequence end to end: each fruit was detected

and classified in real time, commands were transmitted to the controller, staged in per-gate queues, and assigned gates opened at speed-scaled intervals after the fruit reached each gate’s infrared sensor. All four ripening classes were diverted at their respective gates, and defective fruit passed uninterrupted to the reject stream without lost or duplicated commands.

B.5 Shelf-Life Observation Data

The per-class remaining shelf-life values used by the deployed software, listed in Table 4.5, are green 31 days, breaker 29 days, turning 24 days, red 12 days and defect 0 days.

This is the same gap already identified in Section 4.5: the values above match Abekoon et al. (2024)’s published Padma stage durations exactly once mapped through the four-class correspondence of Table 3.6, which is more consistent with the published figures having been carried into the project’s records than with an independent *Padma* 50 green tomatoes replication. Until the underlying raw record is located, or the 31-day trial is repeated and logged, this table should continue to carry that caveat rather than be presented as confirmed independent measurement.

Appendix C: Pin Assignment, Wiring Diagrams and Component Data

A note on the design described. Over the course of the project, three successive control designs were built, of which only the third is currently deployed. The first used single-letter commands and no proximity sensing, with gate timing computed from an assumed constant belt speed; this design was later abandoned because timing error accumulated with distance along the belt. The second, an intermediate development version, introduced numbered class commands but relied on a manually typed class rather than the trained detector, and was never tested end-to-end on the physical rig. The third and current design, described in this appendix and in Section 3.17, uses real infrared sensors to trigger gate actuation and adds a sequence-numbered acknowledgement protocol for reliable delivery over a wireless link. This is the version integrated with the vision system and used to produce the results reported in Chapter 4.

C.1 ESP32 Pin Assignment

Table 5.4 gives the pin assignment of the currently deployed control firmware.

Table 5.4: ESP32 pin assignment, currently deployed firmware

Signal	ESP32 pin
Stepper driver step signal	GPIO 25
Stepper driver direction signal	GPIO 26
Belt-speed potentiometer (analogue input)	GPIO 34
Gate servo 1 (class 1 = green, furthest from camera)	GPIO 13
Gate servo 2 (class 2 = breaker)	GPIO 12
Gate servo 3 (class 3 = turning)	GPIO 14
Gate servo 4 (class 4 = red, closest to camera)	GPIO 27
Infrared sensor, position 1 (co-located with gate 4, not gate 1)	GPIO 18
Infrared sensor, position 2 (co-located with gate 3)	GPIO 19
Infrared sensor, position 3 (co-located with gate 2)	GPIO 21
Infrared sensor, position 4 (co-located with gate 1)	GPIO 23

All four infrared inputs use a pulled-up digital input with a 20 ms software debounce. Servo control uses a 50 Hz pulse-width signal with a 500–2400 μ s pulse range.

On the position/gate numbering mismatch. The infrared sensor position numbers and the servo/gate numbers are deliberately mirrored, not matched – confirmed on the physical rig on 2026-08-15, after an earlier assumption that same-numbered sensors and servos were co-located was found to be incorrect. The confirmed arrangement is: position 1 sits next to gate 4 (class 4, red); position 2 next to gate 3 (turning); position 3 next to gate 2 (breaker); position 4 next to gate 1 (class 1, green). Any wiring diagram or physical rebuild must preserve this mirrored mapping rather than a same-number one.

C.2 Physical Layout

The order of stages along the belt, camera end first, confirmed on the physical rig on 2026-08-15, is: gate 4 (red) → gate 3 (turning) → gate 2 (breaker) → gate 1 (green) – that is, red closest to the camera and green furthest, matching the order stated in Section 3.15. This resolves, in the thesis’s favour, an inconsistency present in an earlier project note dated 2026-07-23, which stated the opposite order (green closest to camera). That earlier note predates the physical confirmation above and should be treated as superseded.

Sensor-to-gate timing. The firmware does not use a measured physical offset, in centimetres, between a sensor and its gate; instead it waits a configurable time after the sensor fires before opening the gate:

- A base post-trigger delay of 2000 ms at the belt’s configured top speed, with a manual fine-tuning offset of –500 ms applied on top, for correcting cases where the computed delay is slightly off in practice.
- Speed-adaptive scaling, enabled by default: both the gate-open duration and the post-trigger delay are scaled against the belt’s current speed relative to its configured top speed (a slower belt gives a shorter gate-open duration and a longer post-trigger delay), within a bounded range so that a near-stopped belt cannot extend the delay indefinitely.
- A default gate-open duration of 2000 ms at top speed, also speed-scaled.
- Gate rest and open angles of 180° and 110° respectively.

All of the values above can be changed at run time over a serial connection, without reprogramming the microcontroller (Section C.4). **[INFORMATION REQUIRED]** No measured physical distance, in centimetres, between the camera, the sensors and the gates was found; only the time-based values above are available.

C.3 Wiring Diagrams

No drawn wiring diagram or schematic image was found for the deployed design; the separation of power domains is documented only as text, consistent with Table 3.2:

- **Motor-driver power domain:** a 24 V, 5 A supply feeds the stepper driver only.
- **Logic power domain:** a separate 12 V, 2 A supply feeds the microcontroller and logic-level circuitry, isolated from the motor and servo rails so that motor and servo switching noise cannot disturb the control electronics (Section 3.15).
- **Servo power:** the four gate servos are signal-wire-only from the microcontroller – powered from a separate 5–6 V supply rather than the microcontroller’s own output pin, which cannot source enough current for four servos and risks resetting the board – with that supply’s ground tied back to the microcontroller’s ground so the signal wires share a common reference.

C.4 Firmware Logic and Command Protocol

The firmware is organised into separate modules for the queue-and-gate control logic, the wireless communication link, one module per gate servo, one module per infrared sensor, the belt motor driver, the belt-speed control input, and a single settings module holding every run-time-tunable value.

Core decision logic. Each classified tomato (a value of 1–4, or 0 for no ripeness class) is placed on a queue as soon as it is classified. Every stage of the belt – there are four, one per gate – is checked on every control-loop pass: if that stage’s sensor has just detected an arriving tomato, the front item is removed from that stage’s queue. If the removed class matches the class handled by that stage, the corresponding gate is scheduled to open after the post-trigger delay described in Appendix C.2, allowing time for the tomato to travel from the sensor to the gate. If the removed class does not match, and the stage is not the last one, the item is placed on the next stage’s queue instead, so that it continues to be tracked until it reaches its own gate. There is no per-tomato

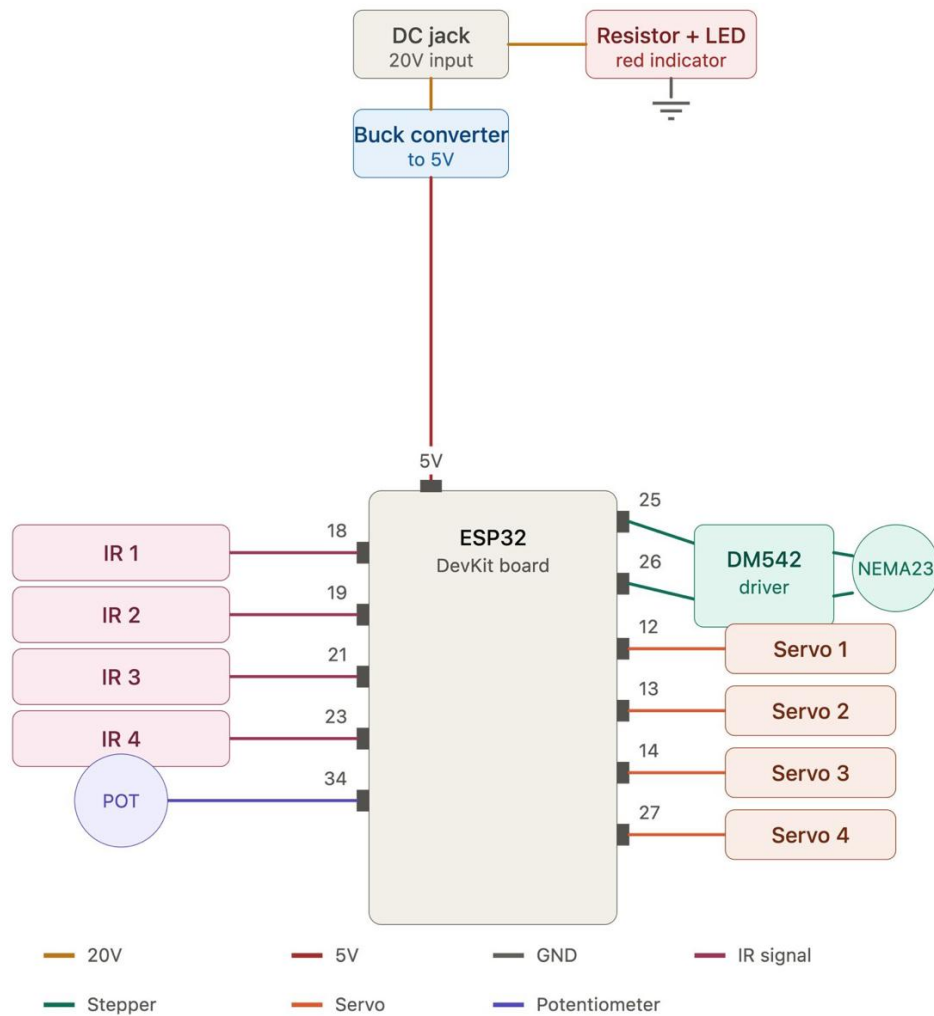


Figure 5.1: Schematic wiring diagram of the ESP32 control system, illustrating GPIO pin configurations, sensor input interfaces, servo/stepper actuator connections, and isolated power distribution domains.

identity beyond this queue position: correctness depends on classifications being sent, in the same order the tomatoes physically enter the belt, before each tomato reaches the first sensor.

Command protocol. Commands are sent as short text messages, over either a wired or a wireless serial connection, using the same handling either way. Table 5.5 lists the commands understood by the firmware.

Table 5.5: Command protocol of the currently deployed firmware

Command	Effect
Class 1–4 (with an optional sequence number)	Places that class on the queue
No-class (with an optional sequence number)	Places a “no gate action” marker on the queue
Reset (with an optional sequence number)	Empties every stage’s queue and clears any gate waiting to open
Set angles	Sets all four servo angles directly
Trigger gate(s)	Opens and returns the listed gate or gates once, for bench testing
Set a configuration value	Changes a run-time setting (for example a servo angle, a gate timing value, or the belt speed range) without reprogramming the microcontroller
Report motor speed	Returns the belt speed currently commanded by the speed-control input

Reliability protocol (sequence number, acknowledgement and retry). The commands that place an item on a queue – the four class commands, the no-class command, and reset – may carry an optional, increasing sequence number. The firmware replies with an acknowledgement that includes the same sequence number and a description of the resulting queue state for every stage. If the same sequence number is received a second time, this is treated as a resend caused by a lost acknowledgement rather than a new command: the firmware repeats the acknowledgement but does not place the tomato on the queue a second time. Reports that the firmware sends on its own initiative, whenever a sensor changes the queue state without being asked, are tagged separately from acknowledgements and carry no sequence number, so that the receiving side can never mistake an unprompted report for a reply to a command it sent.

As currently used, the vision and control software sends class commands without a sequence number, relying on the wireless link's own reliability; the sequence-number and acknowledgement mechanism described above is implemented and working in the firmware and can be connected to a retry mechanism on the software side as a contained addition, should stricter delivery confirmation be required for the closed-loop trial noted in Appendix B.4.

Verification. The firmware was confirmed to build without error and was exercised manually, issuing each command type over a serial connection and confirming the expected servo, sensor and queue-state behaviour, before being connected to the vision system.